

claude-windows / f81fb2b5-e34b-427e-a8a1-a37eb4539a49

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\f81fb2b5-e34b-427e-a8a1-a37eb4539a49\subagents\workflows\wf_56274ff0-603\agent-aeba122d849428319.jsonl`
- SHA-256: `ed8a51e3ab31892766a2c0890b2c3826e4b17c9bb8bcb157dd2ecdf7daeb95bb`
- Source modified: `2026-06-21T02:36:25+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_56274ff0-603`
- session_id: `f81fb2b5-e34b-427e-a8a1-a37eb4539a49`
```

Transcript

1. user (2026-06-21T02:34:25.859Z)

Map the CONTROL PLANE + hosted-service surface. Read backend/main.py (FastAPI endpoints incl. /api/process, /api/jobs, /api/query, /api/compile, /api/highlights, /api/health, CORS config), backend/api/uploads.py (chunked upload for local files; note the <2GB / max_upload_mb limits), backend/api/job_worker.py (async job execution), and the jobs table in backend/infrastructure/database.py. Identify: what ALREADY exists for hosting a service (upload -> process -> jobs -> compile -> highlights -> health), the AUTH gap (CORS '*', no auth), the per-video COST-LEDGER gap (jobs has user_id tenant cols but no cost/price columns), and what's NET-NEW to dispatch a Cloud Run + GPU job (for BOTH segment and stitch) and meter its cost. Tag kind + reuse_status.

2. assistant (2026-06-21T02:34:27.754Z)

I'll map the control plane and hosted-service surface by examining the key backend files. Let me start with parallel reads of the main components.

3. user (2026-06-21T02:34:28.441Z)

```
1      """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.
2
3      Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
4      proxies cap request bodies at ~100 MB ? HOSTING_PLAN $2), are assembled server-side
5      under security.upload_dir, then submitted to /api/process by the returned path.
6      Upload state is in-process by design (this is the single-box alpha): a server restart
7      aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
8      arrives with PR-3 identity scoping.
9
10     The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
11     ``_upload_cfg`` resolves it through the ``backend.main`` module object so it always
12     reads the live value the server set at startup (and so tests that monkeypatch
13     ``backend.main.global_config`` flow through unchanged).
14     """
15     import os
16     import threading
17     import uuid
18     from typing import Any, Dict, Optional
19
20     from fastapi import APIRouter, HTTPException, Request
21     from pydantic import BaseModel
22
23     from backend.utils.hashing import compute_video_id # canonical file-identity hashing
24     from backend.utils.paths import safe_output_filename
25
26     router = APIRouter()
```

```

28     _uploads: Dict[str, Dict[str, Any]] = {}
29     _uploads_lock = threading.Lock()
30
31
32     class UploadInitRequest(BaseModel):
33         filename: str
34         total_size_bytes: Optional[int] = None
35
36
37     class UploadCompleteRequest(BaseModel):
38         total_parts: int
39
40
41     def _upload_cfg():
42         # Resolved lazily (import inside the function) to read the LIVE startup config that
43         # main() sets on the module global, and to avoid a circular import at module load
44         # (main.py imports this router; this module must not import main at import time).
45         import backend.main as _main
46         sec = getattr(_main.global_config, "security", None)
47         upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
48         max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
49         part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
50         exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions",
51 None) or ["mp4", "mov"])]
52         return upload_dir, max_bytes, part_bytes, exts
53
54     def _abort_upload(upload_id: str) -> None:
55         with _uploads_lock:
56             st = _uploads.pop(upload_id, None)
57             if st:
58                 try:
59                     os.remove(st["tmp_path"])
60                 except OSError:
61                     pass
62
63
64     @router.post("/api/upload/init")
65     def upload_init(req: UploadInitRequest):
66         """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
67         upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
68         try:
69             name = safe_output_filename(req.filename)
70         except ValueError as e:
71             raise HTTPException(status_code=400, detail=str(e)) from e
72         ext = os.path.splitext(name)[1].lower().lstrip(".")
73         if ext not in exts:
74             raise HTTPException(status_code=400,
75 detail=f"Extension '{ext}' not allowed. Allowed:
76 {sorted(exts)}")
77         if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
78             raise HTTPException(status_code=413,
79 detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB
80 upload limit.")
81         os.makedirs(upload_dir, exist_ok=True)
82         upload_id = uuid.uuid4().hex
83         tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
84         open(tmp_path, "wb").close()
85         with _uploads_lock:
86             _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0,
87 "bytes": 0}
88         return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
89 "next_part": f"/api/upload/{upload_id}/part/0"}

```

```

89     @router.put("/api/upload/{upload_id}/part/{index}")
90     async def upload_part(upload_id: str, index: int, request: Request):
91         """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
92         _, max_bytes, part_bytes, _ = _upload_cfg()
93         with _uploads_lock:
94             st = _uploads.get(upload_id)
95             if st is None:
96                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
97             if index != st["parts"]:
98                 raise HTTPException(status_code=409,
99                                     detail=f"Out-of-order part: expected {st['parts']}, got
{index} (parts are sequential)")
100
101             received = 0
102             overflow = None
103             with open(st["tmp_path"], "ab") as fh:
104                 async for chunk in request.stream():
105                     received += len(chunk)
106                     if received > part_bytes:
107                         overflow = f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
108                         break
109                     if st["bytes"] + received > max_bytes:
110                         overflow = f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
111                         break
112                     fh.write(chunk)
113             if overflow:
114                 _abort_upload(upload_id)
115                 raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-
init to retry.")
116
117             with _uploads_lock:
118                 st["parts"] += 1
119                 st["bytes"] += received
120             return {"upload_id": upload_id, "received_parts": st["parts"], "received_bytes":
st["bytes"]}
121
122
123     @router.post("/api/upload/{upload_id}/complete")
124     def upload_complete(upload_id: str, req: UploadCompleteRequest):
125         """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
126         upload_dir, _, _, _ = _upload_cfg()
127         with _uploads_lock:
128             st = _uploads.get(upload_id)
129             if st is None:
130                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
131             if st["parts"] == 0 or st["parts"] != req.total_parts:
132                 raise HTTPException(status_code=409,
133                                     detail=f"Expected {req.total_parts} part(s), received
{st['parts']}.")
134             final_path = os.path.join(upload_dir, st["filename"])
135             if os.path.exists(final_path): # never overwrite an earlier upload with the same
name
136                 stem, ext = os.path.splitext(st["filename"])
137                 final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
138             os.replace(st["tmp_path"], final_path)
139             with _uploads_lock:
140                 _uploads.pop(upload_id, None)
141             video_id = compute_video_id(final_path)
142             return {"path": os.path.abspath(final_path), "video_id": video_id,
143                     "size_bytes": os.path.getsize(final_path),
144                     "next": "POST /api/process with this path"}
145

```

```

146
147 @router.delete("/api/upload/{upload_id}")
148 def upload_abort(upload_id: str):
149     """Abort an in-flight upload and delete its partial file."""
150     with _uploads_lock:
151         known = upload_id in _uploads
152         if not known:
153             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
154         _abort_upload(upload_id)
155         return {"status": "aborted", "upload_id": upload_id}
156

```

4. user (2026-06-21T02:34:28.624Z)

```

1     import os
2     import sys
3     import argparse
4     import logging
5     import uuid
6     from urllib.parse import quote
7     import uvicorn
8     from dotenv import load_dotenv
9
10    load_dotenv()
11
12    # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
13    logging.basicConfig(
14        level=logging.INFO,
15        format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
16        datefmt='%H:%M:%S'
17    )
18    logger = logging.getLogger(__name__)
19    from fastapi import FastAPI, HTTPException
20    from fastapi.middleware.cors import CORSMiddleware
21    from pydantic import BaseModel
22    from typing import List, Dict, Any, Optional
23
24    from fastapi.staticfiles import StaticFiles
25    from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
26    from backend.pipeline.validators.base import registry
27    from backend.infrastructure.database import Database
28    from backend.pipeline.segmenters.base import segmenter_registry
29    from backend.pipeline.stitcher.compiler import VideoCompiler
30    from backend.utils.hashing import compute_video_id # canonical file-identity hashing
31    from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
32    from backend.config import load_config as load_app_config, AppConfig, StitchingConfig #
new typed config
33    from backend.results import Failure # standardized error/result contract
34    from backend.reporting import emit_indexer_report
35
36    app = FastAPI(title="Sports Highlight Indexer API")
37
38    # Enable CORS for the local web interface. allow_origins='' with allow_credentials=True
is
39    # rejected by browsers per the fetch spec and is an unsafe default to carry into the
40    # auth/tenancy work; this tool uses no cookies, so credentials stay off.
41    app.add_middleware(
42        CORSMiddleware,
43        allow_origins=["*"],
44        allow_credentials=False,
45        allow_methods=["*"],
46        allow_headers=["*"],
47    )

```

```

48
49 # Serves static frontend files directly (now under api/ per approved structure)
50 os.makedirs("backend/api/static", exist_ok=True)
51 app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
52
53 @app.get("/", response_class=HTMLResponse)
54 def serve_index():
55     index_path = "backend/api/static/index.html"
56     if os.path.exists(index_path):
57         with open(index_path, "r", encoding="utf-8") as f:
58             return f.read()
59     return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
60
61 # Global variables initialized at startup
62 db_instance: Optional[Database] = None
63 global_config: Optional[AppConfig] = None # now typed (Pydantic model)
64
65 # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
66 # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
67 # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
68 # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
69 # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
70 # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
71 # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
72 from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
73     JobWaiting,
74     _arm_wake_timer,
75     _drain_due_jobs,
76     _get_executor,
77     _job_to_request,
78     _MAX_DRAIN_WAKE_SEC,
79     _run_claimed_job,
80     _schedule_next_drain_if_waiting,
81 )
82
83 # Legacy thin wrapper for any remaining direct calls during transition.
84 # New code should import load_app_config / AppConfig from backend.config directly.
85 def load_config(config_path: str = "config.json") -> Dict[str, Any]:
86     cfg = load_app_config(config_path)
87     return cfg.model_dump(mode="json")
88
89 # --- API Models ---
90 class ProcessRequest(BaseModel):
91     video_path: str
92     player_pool: Optional[List[str]] = None
93     max_frames: Optional[int] = None
94     segmenter: Optional[str] = None
95
96 class QueryRequest(BaseModel):
97     video_id: str
98     server: Optional[str] = None
99     receiver: Optional[str] = None
100     duration_min: Optional[float] = None
101     event_type: Optional[str] = None
102
103 class CompileRequest(BaseModel):
104     video_id: str
105     segment_ids: List[int]
106     output_filename: str
107
108 def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:

```

```

109         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
110
111         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
112         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
113         and
114         keep the prior good results intact ? a failed/empty re-run never destroys prior
115         data.
116         """
117         if segments:
118             db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
119         else:
120             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
121
122     # --- API Endpoints ---
123     @app.get("/api/config")
124     def get_config():
125         return global_config
126
127     @app.get("/api/health")
128     def health():
129         """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
130         (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
131         confirm the engine is reachable and which detector is wired. Never raises."""
132         sport = getattr(global_config, "sport", None)
133         indexing = getattr(global_config, "indexing", None)
134         return {
135             "status": "ok",
136             "sport": sport,
137             "default_segmenter": getattr(indexing, "default_segmenter", None),
138         }
139
140     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
141         """The full hash ? validate ? segment ? report pipeline for one video.
142
143         This is the former synchronous /api/process body, unchanged in behavior, now run on
144         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
145         the sync endpoint used to return (UI compatibility); the per-video observability
146         report is still emitted in `finally:` and grafted as response["reports"].
147
148         `progress` is an optional callable(stage: str) used to surface coarse job progress.
149         Raises on unexpected internal errors ? the caller records those as a job failure
150         (after this function has already restored the pre-run segments, see below).
151         """
152         notify = progress or (lambda stage: None)
153
154         notify("hashing")
155         video_id = compute_video_id(req.video_path)
156         was_reingest = db_instance.get_video(video_id) is not None
157
158         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
159         report)
160         notify("validating")
161         logger.info(f"Running validations for {req.video_path}...")
162         val_results, val_context = registry.run_all_with_context(req.video_path,
163         global_config)
164         failures = [r for r in val_results if not r.passed]
165
166         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
167         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
168         "motion")
169         seg_name = req.segmenter or default_seg
170         segmenter_actual = None
171         segmentation_ran = False
172         response: Optional[Dict[str, Any]] = None

```

```

169         try:
170             if failures:
171                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
172                 # status; it no longer destroys the video's prior good segments.
173                 db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
174             response = {
175                 "video_id": video_id,
176                 "status": "failed",
177                 "message": "Video failed validation checks.",
178                 "results": [r.model_dump() for r in val_results],
179             }
180             return response
181
182             # 2. Run segmenter & indexer
183             logger.info("[API] Video passed checks. Segmenting and indexing...")
184             db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
185
186             segmenter_cls = segmenter_registry.get(seg_name)
187             if not segmenter_cls:
188                 # Submit-time validation already 400s unknown names; this is the defensive
189                 # in-worker path (e.g. config default pointing at an unregistered
segmenter).
190                 available = list(segmenter_registry.list_available().keys())
191                 response = {
192                     "video_id": video_id,
193                     "status": "failed",
194                     "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
195                     "results": [r.model_dump() for r in val_results],
196                     "play_segments": [],
197                 }
198                 return response
199
200             logger.info(f"[API] Using segmenter: {seg_name}")
201             notify(f"segmenting ({seg_name})")
202             segmenter_actual = seg_name
203             # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
204             # if the run yields segments; otherwise drop the new partial rows and keep the
old.
205             play_wm, cand_wm = db_instance.segment_watermarks(video_id)
206             segmenter = segmenter_cls(db_instance, global_config)
207             try:
208                 result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
209             except Exception:
210                 # A raising segmenter must not leave half-written rows: restore the pre-run
211                 # state (same destroy-after-confirm contract as the Failure branch), then
let
212                 # the job worker record the failure.
213                 db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
214                 raise
215             segmentation_ran = True
216
217             if isinstance(result, Failure):
218                 logger.error(f"[API] Segmenter failed: {result.message}")
219                 db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
220                 response = {
221                     "video_id": video_id,
222                     "status": "failed",
223                     "message": f"Segmenter '{seg_name}' failed: {result.message}",
224                     "results": [r.model_dump() for r in val_results],
225                     "play_segments": [],

```

```

226         }
227         return response
228
229         segments = result.value if hasattr(result, "value") else result
230         notify("finalizing")
231         _finalize_reingest(video_id, segments, play_wm, cand_wm)
232
233         response = {
234             "video_id": video_id,
235             "status": "success",
236             "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
237             "results": [r.model_dump() for r in val_results],
238             "play_segments": segments,
239         }
240         return response
241     finally:
242         # Always emit the per-video observability report (next to the video, or to
243         # report.output_dir). Never raises. Surface the paths in the response.
244         paths = emit_indexer_report(
245             db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
246             val_results=val_results, val_context=val_context,
247             segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
248             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
249         )
250         if paths and isinstance(response, dict):
251             response["reports"] = paths
252
253
254     @app.post("/api/process", status_code=202)
255     def process_video(req: ProcessRequest):
256         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
257
258         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
259         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
260         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
261         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
262         """
263         allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
264         try:
265             validate_input_path(req.video_path, allowed_root=allowed_root)
266         except ValueError as e:
267             raise HTTPException(status_code=400, detail=str(e)) from e
268         if not os.path.exists(req.video_path):
269             raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
270         if req.segmenter and not segmenter_registry.get(req.segmenter):
271             available = list(segmenter_registry.list_available().keys())
272             raise HTTPException(
273                 status_code=400,
274                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
275             )
276
277         job_id = uuid.uuid4().hex
278         if not db_instance.create_job(job_id, req.video_path, req.segmenter,
279                                     req.player_pool, req.max_frames):
280             raise HTTPException(status_code=500, detail="Failed to persist job record.")
281         _get_executor().submit(_drain_due_jobs)
282         return JSONResponse(
283             status_code=202,
284             content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
285         )

```



```

286
287
288 @app.get("/api/jobs/{job_id}")
289 def get_job(job_id: str):
290     """Job state: {status, progress, result, reports}. `status` = execution state
291     (queued|running|done|failed); the pipeline verdict is result["status"]."""
292     job = db_instance.get_job(job_id)
293     if not job:
294         raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
295     result = job.get("result")
296     return {
297         "job_id": job["id"],
298         "status": job["status"],
299         "progress": job.get("progress"),
300         "video_id": job.get("video_id"),
301         "error": job.get("error"),
302         "result": result,
303         "reports": (result or {}).get("reports"),
304         "created_at": job.get("created_at"),
305         "started_at": job.get("started_at"),
306         "finished_at": job.get("finished_at"),
307     }
308
309 # --- Chunked upload (B2, PR-2) -----
310 # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
311 in
312 # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
313 via
314 # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
315 # sequential-part logic, and abort cleanup ? is unchanged.
316 from backend.api import uploads as uploads_api
317
318 app.include_router(uploads_api.router)
319
320 # --- Highlight download (B3, PR-2) -----
321
322 @app.get("/api/highlights/{video_id}/{filename}")
323 def download_highlight(video_id: str, filename: str):
324     """Stream a compiled highlight reel ? `filename` is the bare name given to
325     /api/compile
326     (which only writes into output_dir; the browser previously got back an unreachable
327     server-local path)."""
328     try:
329         name = safe_output_filename(filename)
330     except ValueError as e:
331         raise HTTPException(status_code=400, detail=str(e)) from e
332     if not db_instance.get_video(video_id):
333         raise HTTPException(status_code=404, detail="Indexed video record not found.")
334     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
335     or "output"
336     path = os.path.join(output_dir, name)
337     try:
338         path = resolve_under_root(path, output_dir)
339     except ValueError as e:
340         raise HTTPException(status_code=400, detail=str(e)) from e
341     if not os.path.isfile(path):
342         raise HTTPException(status_code=404,
343                             detail=f"No compiled file named '{name}'. Compile first via
344                             /api/compile.")
345     media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
346     return FileResponse(path, media_type=media, filename=name)
347
348 @app.post("/api/query")

```

```

346 def query_play_segments(req: QueryRequest):
347     filters = {
348         "server": req.server,
349         "receiver": req.receiver,
350         "duration_min": req.duration_min,
351         "event_type": req.event_type
352     }
353     segments = db_instance.query_play_segments(req.video_id, filters)
354     return {"play_segments": segments}
355
356 @app.post("/api/compile")
357 def compile_highlights(req: CompileRequest):
358     video = db_instance.get_video(req.video_id)
359     if not video:
360         raise HTTPException(status_code=404, detail="Indexed video record not found.")
361
362     # Retrieve matching play segments from db
363     all_segments = db_instance.query_play_segments(req.video_id, {})
364     matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
365
366     if not matched_segments:
367         raise HTTPException(status_code=400, detail="No matching play segments found to
368 stitch.")
369
370     # Reject path traversal / absolute output names (os.path.join would otherwise let an
371     # absolute or '..'-laden output_filename write anywhere on disk).
372     try:
373         out_name = safe_output_filename(req.output_filename)
374     except ValueError as e:
375         raise HTTPException(status_code=400, detail=str(e)) from e
376
377     # The configured shot-count floor (stitching.min_shot_count) applies on the API
378     # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
379     # metadata are exempt: the floor filters known counts only, so explicitly
380     # requested manual/legacy segments can't silently vanish under the default floor.
381     stitching = getattr(global_config, "stitching", None) or StitchingConfig()
382     kept = []
383     for s in matched_segments:
384         meta = s.get("metadata") or {}
385         sc = meta.get("shot_count") if isinstance(meta, dict) else None
386         try:
387             if sc is not None and int(sc) < stitching.min_shot_count:
388                 continue
389             except (TypeError, ValueError):
390                 pass # unparseable count -> treat as unknown, keep
391             kept.append(s)
392
393     if len(kept) < len(matched_segments):
394         logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
395 dropped "
396 f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
397 segments below the floor.")
398         if not kept:
399             raise HTTPException(status_code=400,
400 detail=f"All {len(matched_segments)} matching segments fall
401 below "
402 f"stitching.min_shot_count={stitching.min_shot_count}.")
403
404     # Extract start/end time pairs
405     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
406
407     # Run compiler with the configured stitching paddings + optional branding watermark
408     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
409     or "output"

```

```

404         compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
405                                   end_padding=stitching.end_padding,
watermark=stitching.watermark)
406         try:
407             out_path = compiler.compile_highlights(video["filepath"], time_pairs, out_name)
408             # `filepath` is server-local (not reachable by the browser); also return the
ready-to-use
409             # download URL so the SPA needn't hand-build it (P6 / PER_RALLY_SELECTION.md
§4.3).
410             download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
411             return {"status": "success", "filepath": out_path, "download_url": download_url}
412         except Exception as e:
413             raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
414
415     # --- CLI Debug Utility & Orchestration ---
416     def run_cli_diagnose_clip(video_path: str, timestamp: float):
417         """CLI debugging utility to examine segment properties around a timestamp."""
418         print("-" * 60)
419         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
420         print("-" * 60)
421
422         cfg = load_config() # legacy wrapper still works, returns dict for now
423
424         # 1. Validation check diagnostics
425         print("[DIAGNOSTIC] Running validation framework...")
426         results = registry.run_all(video_path, cfg)
427         for r in results:
428             status_symbol = "[PASS]" if r.passed else "[FAIL]"
429             print(f"    {status_symbol} {r.validator_name}: {r.message}")
430             if r.details:
431                 print(f"        Details: {r.details}")
432
433         # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
434         print("-" * 60)
435         print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
436         import cv2
437         cap = cv2.VideoCapture(video_path)
438         if not cap.isOpened():
439             print("[FAIL] OpenCV could not open video.")
440             return
441
442         fps = cap.get(cv2.CAP_PROP_FPS)
443         total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
444
445         start_frame = max(0, int((timestamp - 3.0) * fps))
446         end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
447
448         # Measure frame-by-frame changes in sample region
449         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
450         prev_gray = None
451         motions = []
452
453         for _ in range(start_frame, end_frame):
454             ret, frame = cap.read()
455             if not ret:
456                 break
457             gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
458             gray = cv2.GaussianBlur(gray, (21, 21), 0)
459
460             if prev_gray is not None:
461                 diff = cv2.absdiff(prev_gray, gray)
462                 _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
463                 motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])

```

```

464         motions.append(motion_ratio)
465         prev_gray = gray
466
467     cap.release()
468
469     if motions:
470         avg_motion = sum(motions) / len(motions)
471         # Support both legacy dict (from wrapper) and future direct model
472         if hasattr(cfg, "indexing"):
473             threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
474         else:
475             threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08) # type:
ignore[attr-defined]
476         print(f" Sample Frame Count: {len(motions)}")
477         print(f" Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
478         if avg_motion > threshold:
479             print(" Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
480         else:
481             print(" Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
482         else:
483             print(" [ERROR] No visual frames were extracted in the sample range.")
484
485     print("=" * 60)
486
487     def main():
488         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
489         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
490         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
491         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
492         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
493         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
494         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
495         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
496
497         available_segmenters = list(segmenter_registry.list_available().keys())
498         parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
499         parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
500         parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
501         parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
502
503         args = parser.parse_args()
504
505         if args.setup:
506             # Invoke the diagnostics/bootstrap script (renamed from the old root
507             # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
508             # build system). Resolve its path from this file so `indexer --setup`
509             # works regardless of the caller's cwd.
510             import subprocess
511             doctor_path = os.path.join(
512                 os.path.dirname(os.path.abspath(__file__)),
513                 "scripts", "doctor.py",

```

```

514         )
515         print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
516         subprocess.run([sys.executable, doctor_path])
517         return
518
519     if args.trim_start is not None and args.trim_end is not None:
520         if not args.video_path:
521             print("[ERROR] Please provide --video-path to extract a segment.")
522             return
523
524         # Determine output path
525         out_filename = args.trim_output or "trimmed_segment.mp4"
526         if os.path.isabs(out_filename):
527             out_path = out_filename
528             out_dir = os.path.dirname(out_path)
529             out_name = os.path.basename(out_path)
530         else:
531             out_dir = args.output_dir
532             out_path = os.path.join(out_dir, out_filename)
533             out_name = out_filename
534
535         os.makedirs(out_dir, exist_ok=True)
536         print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
537
538         # global_config isn't initialized this early; load the typed config so the trim
539         # clip carries the same configured stitching paddings + watermark as a real
540         # highlight (watermark default-on).
541         stitching = load_app_config().stitching
542         compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
543                                 end_padding=stitching.end_padding,
544                                 watermark=stitching.watermark)
545         try:
546             res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
547 args.trim_end)], out_name)
548             print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
549         except Exception as e:
550             print(f"[ERROR] Failed to compile trimmed segment: {e}")
551         return
552
553     if args.debug_clip is not None:
554         if not args.video_path:
555             print("[ERROR] Please provide --video-path to debug a specific clip.")
556             return
557         run_cli_diagnose_clip(args.video_path, args.debug_clip)
558         return
559
560     # Initialize Global Config and DB
561     global global_config, db_instance
562     global_config = load_app_config() # returns typed AppConfig
563
564     # The CLI --output-dir overrides the configured output directory. It now lives
565     # on the typed model (OutputConfig.output_dir), so every consumer ? including
566     # /api/compile ? reads the same value instead of a write-only runtime hack.
567     global_config.output.output_dir = args.output_dir
568     os.makedirs(global_config.output.output_dir, exist_ok=True)
569
570     db_path = os.path.join(global_config.output.output_dir,
571 global_config.output.db_name)
572     db_instance = Database(db_path)
573
574     # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
575     # the executor is in-process. Mark them failed so pollers see a terminal state.
576     swept = db_instance.fail_stale_jobs()
577     if swept:

```

```

575         logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
576
577     # CLI processing mode (no web UI needed)
578     if args.video_path and not args.server:
579         print(f"[CLI] Processing video file: {args.video_path}")
580         if not os.path.exists(args.video_path):
581             print(f"[FAIL] Video file not found: {args.video_path}")
582             return
583
584         video_id = compute_video_id(args.video_path)
585         was_reingest = db_instance.get_video(video_id) is not None
586
587         # Validate (with-context variant surfaces ffprobe_meta for the report)
588         print(f"[CLI] Validating...")
589         val_results, val_context = registry.run_all_with_context(args.video_path,
global_config)
590         failures = [r for r in val_results if not r.passed]
591
592         # Save video status
593         status = "failed" if failures else "processed"
594         db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])
595
596         seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
597         segmenter_actual = None
598         segmentation_ran = False
599         try:
600             print("\n" + "="*40)
601             print("VALIDATION SUMMARY")
602             print("="*40)
603             for r in val_results:
604                 status_symbol = "[PASS]" if r.passed else "[FAIL]"
605                 print(f"{status_symbol} {r.validator_name}: {r.message}")
606             print("="*40 + "\n")
607
608             if failures:
609                 print("[FAIL] Video failed checks. Indexing halted.")
610                 return
611
612             # Index
613             segmenter_cls = segmenter_registry.get(seg_name)
614             if not segmenter_cls:
615                 print(f"[FAIL] Segmenter '{seg_name}' not found.")
616                 return
617
618             print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
619             segmenter_actual = seg_name
620             play_wm, cand_wm = db_instance.segment_watermarks(video_id)
621             segmenter = segmenter_cls(db_instance, global_config)
622             result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
623             segmentation_ran = True
624
625             if isinstance(result, Failure):
626                 print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
627                 print("[FAIL] Indexing halted for video.")
628                 segments = []
629             else:
630                 segments = result.value if hasattr(result, "value") else result
631                 print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
632                 # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.

```

```

633         _finalize_reingest(video_id, segments, play_wm, cand_wm)
634     finally:
635         # Always emit the per-video observability report. Never raises.
636         paths = emit_indexer_report(
637             db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
638             val_results=val_results, val_context=val_context,
639             segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
640             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
641         )
642     if paths:
643         print(f"[REPORT] Technical report : {paths.get('technical_md')}")
644         print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
645         print(f"[REPORT] Machine JSON      : {paths.get('json')}")
646     return
647
648     # Start Local Server Mode
649     if args.server or not args.video_path:
650         print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
651         # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
652         # in-process wake-timer for any parked 'waiting' job died with the old process,
so
653         # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
654         _get_executor().submit(_drain_due_jobs)
655         uvicorn.run(app, host="127.0.0.1", port=args.port)
656
657 if __name__ == "__main__":
658     main()
659

```

5. user (2026-06-21T02:34:28.642Z)

```

1     """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3).
2
3     Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
4     The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
5     that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
6     re-claimed when due ? with no central scheduler (the DB is the clock).
7
8     The monkeypatch contract these functions rely on (and that tests depend on):
9     every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
10    `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
11    `backend.main` module object at call-time (`import backend.main as _main`), never as a
12    bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
13    ...) ` still intercepts the inter-function calls (e.g. a parked job arms the `patched`
14    `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
15    inside the functions also avoids the circular import at module load (main.py imports
this
16    module; this module must not import main at import time).
17    """
18    import logging
19    import threading
20    import traceback
21    from concurrent.futures import ThreadPoolExecutor
22    from typing import Any, Dict, Optional
23
24    logger = logging.getLogger(__name__)
25
26    # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
27    # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
28    # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters

```

```

29     # already parallelize their AI handoff internally via their own pools.
30     _job_executor: Optional[ThreadPoolExecutor] = None
31
32
33     def _get_executor() -> ThreadPoolExecutor:
34         """Lazy module-global executor; tests monkeypatch this for inline execution."""
35         global _job_executor
36         if _job_executor is None:
37             _job_executor = ThreadPoolExecutor(max_workers=1,
thread_name_prefix="jobworker")
38         return _job_executor
39
40
41     class JobWaiting(Exception):
42         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
43         WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`,
44         NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
45         releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
46         is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.
47
48         The wiring is additive: the production segmenter path never raises this today (zero
behaviour change), so a job runs exactly as before. The hook is what a later slice
49         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
50         """
51
52
53         def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
54             super().__init__(f"job parked for {delay_sec:.0f}s")
55             self.delay_sec = max(0.0, float(delay_sec))
56             self.progress = progress
57
58
59     def _job_to_request(job: Dict[str, Any]):
60         """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
61         re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
62         the original submit. The row stores exactly the ProcessRequest fields."""
63         import backend.main as _main
64         return _main.ProcessRequest(
65             video_path=job["video_path"],
66             player_pool=job.get("player_pool"),
67             max_frames=job.get("max_frames"),
68             segmenter=job.get("segmenter"),
69         )
70
71
72     def _run_claimed_job(job: Dict[str, Any]) -> None:
73         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
74         terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
75
76         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
77         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
78         result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
79         job self-scheduled and will be re-claimed when `next_poll_at` is due.
80         """
81         import backend.main as _main
82         db_instance = _main.db_instance
83         job_id = job["id"]
84         req = _main._job_to_request(job)
85         try:
86             result = _main._execute_processing(

```



```

87         req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
88     if result.get("video_id"):
89         db_instance.set_job_video_id(job_id, result["video_id"])
90     db_instance.mark_job_done(job_id, result)
91 except _main.JobWaiting as w:
92     # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
93     logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
94     db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
95 except Exception as e:
96     logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
97     db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
98
99
100 def _drain_due_jobs() -> int:
101     """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
102     `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
103
104     Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
105     up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
106     a single-worker box keeps making progress with no central timer (the DB is the
clock).
107
108     Returns the number of jobs that reached a terminal/parked state this tick.
109     """
110     import backend.main as _main
111     ran = 0
112     while True:
113         job = _main.db_instance.claim_due_job()
114         if job is None:
115             break
116         ran += 1
117         _main._run_claimed_job(job)
118     # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
119     # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
120     # on the next submit/drain. Best-effort ? never raises into the worker.
121     _main._schedule_next_drain_if_waiting()
122     return ran
123
124 def _schedule_next_drain_if_waiting() -> None:
125     """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
126     self-scheduled job resumes with no further client submit. The drain itself re-checks
127     due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
128     import backend.main as _main
129     try:
130         delay = _main.db_instance.seconds_until_next_due()
131     except Exception as e: # pragma: no cover - defensive; never wedge the worker
132         logger.error(f"Could not compute next-due delay: {e}")
133         return
134     if delay is None:
135         return # nothing waiting
136     # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
137     _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)))
138
139
140 def _arm_wake_timer(delay_sec: float) -> None:
141     """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
142

```

```

can
143 monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting. """
144 import backend.main as _main
145 timer = threading.Timer(delay_sec, lambda:
_main._get_executor().submit(_main._drain_due_jobs))
146 timer.daemon = True
147 timer.start()
148
149
150 #: A parked job further out than this is re-checked by a capped wake rather than one
long
151 #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
152 _MAX_DRAIN_WAKE_SEC = 60.0
153

```

6. user (2026-06-21T02:34:29.159Z)

```

1 import sqlite3
2 import json
3 import logging
4 from contextlib import contextmanager
5 from typing import List, Dict, Any, Optional, Tuple
6
7 logger = logging.getLogger(__name__)
8
9 class Database:
10     def __init__(self, db_path: str):
11         self.db_path = db_path
12         self._init_db()
13
14     @contextmanager
15     def _connect(self):
16         """Transaction scope that also CLOSES the connection.
17
18         ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
19         never closes, so every call leaked its connection to GC (lingering file
20         handles on the WAL sidecars under Windows). Internal methods use this.
21         """
22         conn = self._get_connection()
23         try:
24             with conn:
25                 yield conn
26         finally:
27             conn.close()
28
29     def _get_connection(self):
30         # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
31         # adds concurrent writers (a swallowed 'database is locked' silently drops
segments).
32         conn = sqlite3.connect(self.db_path, timeout=30.0)
33         conn.row_factory = sqlite3.Row
34         # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
35         conn.execute("PRAGMA foreign_keys = ON")
36         # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
37         conn.execute("PRAGMA busy_timeout = 30000")
38         try:
39             conn.execute("PRAGMA journal_mode = WAL")
40         except sqlite3.OperationalError:
41             pass # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
42         return conn
43
44     def _execute_write(self, sql: str, params: Tuple[Any, ...], error_msg: str) -> bool:
45         """Run ONE bool-returning single-statement write inside a connection scope.
46

```

```

47         Captures the connect/commit/except-logger.error/return-False boilerplate shared
48         by
49         the bool-returning single-statement writers. ``error_msg`` is the caller's exact
50         per-method message (already formatted with its own ids) ? logged verbatim on a
51         swallowed write fault so each writer keeps its specific log text. Returns True
52         on
53         success, False (after logging ``error_msg``) on any exception.""
54     try:
55         with self._connect() as conn:
56             conn.cursor().execute(sql, params)
57             conn.commit()
58             return True
59     except Exception as e:
60         logger.error(f"{error_msg}: {e}")
61         return False
62
63 @staticmethod
64 def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
65     """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runably.
66
67     A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn``
68     transaction, so an interrupted v3/v4 block can leave the column added while
69     ``user_version`` stays un-bumped ? and the next open would re-run the same
70     ``ALTER`` and raise ``OperationalError: duplicate column name``, bricking every
71     future ``Database(...)`` construction (``_init_db`` runs in ``__init__``).
72
73     Swallow
74     ONLY that specific error so the ladder stays crash-safe, matching the re-
75     runnable
76     ``CREATE ... IF NOT EXISTS`` v1/v2/v5 blocks.
77     """
78     try:
79         cursor.execute(ddl)
80     except sqlite3.OperationalError as exc:
81         if "duplicate column name" not in str(exc).lower():
82             raise
83
84 def _init_db(self):
85     """Initializes tables for videos, play_segments, and events."""
86     with self._connect() as conn:
87         cursor = conn.cursor()
88
89         self._create_base_tables(cursor)
90
91         # Versioned migrations live here. PRAGMA user_version is the schema
92         # contract ? bump it for every change and add an ordered `if version < N`
93         # block below. Tests assert the expected version, so accidental drift fails
94         CI.
95
96         version = cursor.execute("PRAGMA user_version").fetchone()[0]
97
98         if version < 1:
99             self._migrate_to_v1(cursor)
100
101         if version < 2:
102             self._migrate_to_v2(cursor)
103
104         if version < 3:
105             self._migrate_to_v3(cursor)
106
107         if version < 4:
108             self._migrate_to_v4(cursor)
109
110         if version < 5:
111             self._migrate_to_v5(cursor)
112
113         # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA
114         user_version = 6

```

```

106
107         conn.commit()
108
109     def _create_base_tables(self, cursor):
110         """Create the videos / play_segments / events base tables (idempotent)."""
111         # Videos Table
112         cursor.execute("""
113             CREATE TABLE IF NOT EXISTS videos (
114                 id TEXT PRIMARY KEY,
115                 filepath TEXT NOT NULL,
116                 processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
117                 status TEXT NOT NULL,
118                 validation_results TEXT
119             )
120         """)
121
122         # Play Segments Table (Extensible metadata JSON)
123         cursor.execute("""
124             CREATE TABLE IF NOT EXISTS play_segments (
125                 id INTEGER PRIMARY KEY AUTOINCREMENT,
126                 video_id TEXT NOT NULL,
127                 start_time REAL NOT NULL,
128                 end_time REAL NOT NULL,
129                 duration REAL NOT NULL,
130                 server TEXT,
131                 receiver TEXT,
132                 metadata TEXT,
133                 FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
134             )
135         """)
136
137         # Events Table
138         cursor.execute("""
139             CREATE TABLE IF NOT EXISTS events (
140                 id INTEGER PRIMARY KEY AUTOINCREMENT,
141                 segment_id INTEGER NOT NULL,
142                 timestamp REAL NOT NULL,
143                 type TEXT NOT NULL,
144                 player TEXT,
145                 details TEXT,
146                 FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE CASCADE
147             )
148         """)
149
150     def _migrate_to_v1(self, cursor):
151         # v1: raw candidate detections (pre-confirmation), detector-agnostic.
152         # Common fields are columns; detector-specific metrics go in `stats` JSON,
153         # so a new segmenter reuses this table by setting its own `detector`/`stats`.
154         cursor.execute("""
155             CREATE TABLE IF NOT EXISTS candidate_segments (
156                 id INTEGER PRIMARY KEY AUTOINCREMENT,
157                 video_id TEXT NOT NULL,
158                 detector TEXT NOT NULL,
159                 chunk_index INTEGER,
160                 start_time REAL NOT NULL,
161                 end_time REAL NOT NULL,
162                 duration REAL NOT NULL,
163                 confidence REAL,
164                 stats TEXT,
165                 detection_params TEXT,
166                 confirmed_segment_id INTEGER,
167                 human_label TEXT,
168                 notes TEXT,
169                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
170                 FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,

```

```

171         FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON
DELETE SET NULL
172     )
173     """)
174     cursor.execute("""
175         CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
176         ON candidate_segments(video_id, detector)
177     """)
178     cursor.execute("PRAGMA user_version = 1")
179
180     def _migrate_to_v2(self, cursor):
181         # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per submitted
182         # /api/process request. `status` is the EXECUTION state of the job
183         # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
184         # that failed validation) lives inside `result` JSON as result["status"].
185         # `result` is the full sync-era response dict (incl. report paths), so the
186         # observability report is the job's artifact, per the plan.
187         cursor.execute("""
188             CREATE TABLE IF NOT EXISTS jobs (
189                 id TEXT PRIMARY KEY,
190                 video_path TEXT NOT NULL,
191                 video_id TEXT,
192                 segmenter TEXT,
193                 player_pool TEXT,
194                 max_frames INTEGER,
195                 status TEXT NOT NULL DEFAULT 'queued',
196                 progress TEXT,
197                 error TEXT,
198                 result TEXT,
199                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
200                 started_at TIMESTAMP,
201                 finished_at TIMESTAMP
202             )
203             """)
204         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON jobs(status)")
205         cursor.execute("PRAGMA user_version = 2")
206
207     def _migrate_to_v3(self, cursor):
208         # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2). `policy` = the
209         # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting' job is
210         # due
211         # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ? any worker
212         # re-claims it via `claim_due_job` when due ? with NO master node: the table IS
213         # the coordinator. ALTER (not recreate) so existing v2 job rows survive; the
214         # ADD COLUMNS are applied idempotently so an interrupted upgrade can re-run
215         # them.
216         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy TEXT")
217         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at
TIMESTAMP")
218         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
219         cursor.execute("PRAGMA user_version = 3")
220
221     def _migrate_to_v4(self, cursor):
222         # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a NULLABLE
223         # `user_id` to videos + candidate_segments so a future per-user / multi-tenant
224         # path can scope rows to an owner. **NULL = local install = byte-identical to
225         # today** ? every existing row stays NULL and every existing query (which never
226         # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows survive;
227         # the ADD COLUMNS are applied idempotently so an interrupted upgrade can re-run
228         # them.
229         # Additive ONLY: no served-behavior change rides on this slice.
230         self._add_column_idempotent(cursor, "ALTER TABLE videos ADD COLUMN user_id
TEXT")
231         self._add_column_idempotent(cursor, "ALTER TABLE candidate_segments ADD COLUMN

```

```

user_id TEXT")
229         # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast path free
230         # of index churn while making the eventual per-user lookups cheap.
231         cursor.execute(
232             "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
233             "WHERE user_id IS NOT NULL")
234         cursor.execute(
235             "CREATE INDEX IF NOT EXISTS idx_candidates_user "
236             "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
237         cursor.execute("PRAGMA user_version = 4")
238
239     def _migrate_to_v5(self, cursor):
240         # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per (user_id,
241         # version): the resolved per-user `fusion_config` overlay + an INLINE
242         # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
evidence
243         # and provenance that earned it. `is_active` pins the one row the serving bridge
244         # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`). This
slice
245         # only persists/loads it ? NOTHING reads it on the served path yet.
246         #
247         #   user_id           ? owner of the model (NULL allowed = the local install)
248         #   version           ? monotonic per-user model version
249         #   baseline_version  ? the shared baseline this was fit against (upgrade
switch)
250         #   feature_schema_hash ? fingerprint of the feature set; MINOR-vs-MAJOR re-fit
gate
251         #   fusion_config     ? JSON per-user FusionConfig overlay (cue_flags /
thresholds)
252         #   classifier_json   ? JSON LogisticRegression.to_dict() (None = config-only
model)
253         #   promotion_evidence ? JSON per_user_gate PromotionDecision (why it was
promoted)
254         #   n_videos_at_fit   ? held-out-user corpus size when fit (min_n trust floor
input)
255         #   is_active         ? the one resolved row for this user (partial-unique
below)
256         cursor.execute("""
257             CREATE TABLE IF NOT EXISTS user_models (
258                 id INTEGER PRIMARY KEY AUTOINCREMENT,
259                 user_id TEXT,
260                 version INTEGER NOT NULL DEFAULT 1,
261                 baseline_version TEXT,
262                 feature_schema_hash TEXT,
263                 fusion_config TEXT,
264                 classifier_json TEXT,
265                 promotion_evidence TEXT,
266                 n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
267                 is_active INTEGER NOT NULL DEFAULT 0,
268                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
269             )
270         """)
271         # One model version per user (NULLs compare distinct in SQLite, so the local
272         # install can still carry multiple versioned rows; the active-row guard below
273         # is what enforces a single served model).
274         cursor.execute("""
275             CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
276             ON user_models(user_id, version)
277         """)
278         # At most ONE active model per user: a partial-unique index over the active
rows.
279         # (SQLite treats NULL user_id rows as distinct, so the local install's single
280         # active row is still guarded ? there is one local user.)
281         cursor.execute("""
282             CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active

```

```

283         ON user_models(user_id) WHERE is_active = 1
284     """)
285     cursor.execute("PRAGMA user_version = 5")
286
287     def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
288         """Register or update a video row WITHOUT touching its child segments.
289
290         Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
291         with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
292         That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
293         before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
294         row in place; clearing segments is now an explicit, deliberate operation
295         (clear_video_data / prune_segments).
296         """
297         return self._execute_write(
298             "INSERT INTO videos (id, filepath, status, validation_results) VALUES (?, ?,
?, ?) "
299             "ON CONFLICT(id) DO UPDATE SET "
300             "filepath=excluded.filepath, status=excluded.status, "
301             "validation_results=excluded.validation_results",
302             (video_id, filepath, status, json.dumps(validation_results)),
303             "Failed to add video",
304         )
305
306     def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
307         """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).
308
309         Captured before a (re)segmentation so the run's new rows (id > watermark) can be
310         told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
311         """
312         with self._connect() as conn:
313             cur = conn.cursor()
314             p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
315                             (video_id,)).fetchone()[0]
316             c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
317                             (video_id,)).fetchone()[0]
318             return int(p), int(c)
319
320     def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
321                       play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
322                       cand_after: Optional[int] = None) -> None:
323         """Delete play/candidate segments by id range (events cascade from
play_segments).
324
325         Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
326         successful run (`*_upto` = the pre-run watermark), or drop the NEW partial
rows
327         and keep the OLD ones when the run failed/produced nothing (`*_after`).
328         """
329         with self._connect() as conn:
330             cur = conn.cursor()
331             if play_upto is not None:
332                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
333             if play_after is not None:
334                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))

```

```

335         if cand_upto is not None:
336             cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
? ", (video_id, cand_upto))
337         if cand_after is not None:
338             cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
? ", (video_id, cand_after))
339         conn.commit()
340
341     def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
342         try:
343             with self._connect() as conn:
344                 cursor = conn.cursor()
345                 cursor.execute(
346                     "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
347                     (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {})))
348             )
349             conn.commit()
350             return cursor.lastrowid
351         except Exception as e:
352             logger.error(f"Failed to add play segment: {e}")
353             return None
354
355     def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
356         return self._execute_write(
357             "INSERT INTO events (segment_id, timestamp, type, player, details) VALUES
(?, ?, ?, ?, ?)",
358             (segment_id, timestamp, event_type, player, json.dumps(details or {})),
359             "Failed to add event",
360         )
361
362     def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
363                               chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
364                               stats: Optional[Dict[str, Any]] = None,
365                               detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
366         """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
367         detector-specific dicts stored as JSON. Returns the new candidate id, or
368         None."""
369         try:
370             with self._connect() as conn:
371                 cursor = conn.cursor()
372                 cursor.execute(
373                     """INSERT INTO candidate_segments
374                     (video_id, detector, chunk_index, start_time, end_time, duration,
375                     confidence, stats, detection_params)
376                     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
377                     (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
378                     confidence, json.dumps(stats or {}), json.dumps(detection_params or
{})))
379                 conn.commit()
380                 return cursor.lastrowid
381         except Exception as e:
382             logger.error(f"Failed to add candidate segment: {e}")
383             return None
384
385     def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:

```



```

386         """Record that a candidate detection was confirmed as a given play_segment."""
387     return self._execute_write(
388         "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id = ?",
389         (segment_id, candidate_id),
390         f"Failed to link candidate {candidate_id} to segment {segment_id}",
391     )
392
393     def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
394                                only_unlabeled: bool = False) -> List[Dict[str, Any]]:
395         """Fetch candidate detections for the annotation / fine-tuning workflow.
396         `stats` and `detection_params` are deserialized from JSON."""
397         query = "SELECT * FROM candidate_segments WHERE video_id = ?"
398         params: List[Any] = [video_id]
399
400         if detector:
401             query += " AND detector = ?"
402             params.append(detector)
403         if only_unlabeled:
404             query += " AND human_label IS NULL"
405
406         query += " ORDER BY start_time"
407
408         candidates = []
409         with self._connect() as conn:
410             rows = conn.cursor().execute(query, params).fetchall()
411             for row in rows:
412                 d = dict(row)
413                 d["stats"] = json.loads(d["stats"] or "{}")
414                 d["detection_params"] = json.loads(d["detection_params"] or "{}")
415                 candidates.append(d)
416         return candidates
417
418     def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
419         with self._connect() as conn:
420             row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
421 (video_id,)).fetchone()
422             if row:
423                 d = dict(row)
424                 d["validation_results"] = json.loads(d["validation_results"] or "[]")
425                 return d
426         return None
427
428     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
429         """
430         Dynamically queries play segments based on filters:
431         - duration_min: float
432         - server: string
433         - receiver: string
434         - event_type: string (e.g. smash, foul)
435         """
436         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
437         params = [video_id]
438
439         if filters.get("duration_min"):
440             query += " AND r.duration >= ?"
441             params.append(filters["duration_min"])
442
443         if filters.get("server"):
444             query += " AND r.server = ?"
445             params.append(filters["server"])
446
447         if filters.get("receiver"):
448             query += " AND r.receiver = ?"
449             params.append(filters["receiver"])

```

```

449
450         if filters.get("event_type"):
451             # Check if there is an event of a specific type inside this segment
452             query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
453             params.append(filters["event_type"])
454
455         segments_list = []
456         with self._connect() as conn:
457             cursor = conn.cursor()
458             rows = cursor.execute(query, params).fetchall()
459
460             for row in rows:
461                 r_dict = dict(row)
462                 r_id = r_dict["id"]
463                 r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
464
465                 # Fetch associated events
466                 event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
467                 r_dict["events"] = []
468                 for er in event_rows:
469                     e_dict = dict(er)
470                     e_dict["details"] = json.loads(e_dict["details"] or "{}")
471                     r_dict["events"].append(e_dict)
472
473                 segments_list.append(r_dict)
474
475         return segments_list
476
477     def clear_video_data(self, video_id: str):
478         with self._connect() as conn:
479             conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
480             conn.commit()
481
482     # --- Async job store (DEFERRED_HARDENING_PLAN #16) -----
483     # Writers follow the repo convention (swallow + logger.error, return bool); the
484     # worker logs loudly on a failed transition so a job can't silently wedge.
485
486     def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
487                   player_pool: Optional[List[str]] = None,
488                   max_frames: Optional[int] = None,
489                   policy: Optional[Dict[str, Any]] = None) -> bool:
490         """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
491         (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy."""
492         return self._execute_write(
493             "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames,
status, policy) "
494             "VALUES (?, ?, ?, ?, ?, 'queued', ?)",
495             (job_id, video_path, segmenter,
496              json.dumps(player_pool) if player_pool is not None else None,
497              max_frames, json.dumps(policy) if policy is not None else None),
498             f"Failed to create job {job_id}",
499         )
500
501     def mark_job_running(self, job_id: str) -> bool:
502         return self._execute_write(
503             "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
504             "progress='starting' WHERE id = ?", (job_id,),
505             f"Failed to mark job {job_id} running",
506         )
507
508     def set_job_progress(self, job_id: str, progress: str) -> bool:
509         return self._execute_write(

```

```

510         "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id),
511         f"Failed to set progress for job {job_id}",
512     )
513
514     def set_job_video_id(self, job_id: str, video_id: str) -> bool:
515         return self._execute_write(
516             "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id),
517             f"Failed to set video_id for job {job_id}",
518         )
519
520     def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
521         """Job EXECUTED to completion. The pipeline's own verdict (success/failed
522         validation) is inside `result` ? job status 'done' means 'the worker
523         finished'."""
524         return self._execute_write(
525             "UPDATE jobs SET status='done', progress='finished', result=?, "
526             "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
527             (json.dumps(result), job_id),
528             f"Failed to mark job {job_id} done",
529         )
530
531     def mark_job_failed(self, job_id: str, error: str) -> bool:
532         """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
533         return self._execute_write(
534             "UPDATE jobs SET status='failed', error=?, "
535             "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id),
536             f"Failed to mark job {job_id} failed",
537         )
538
539     def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
540         """Fetch one job; `result`/`player_pool` JSON deserialized."""
541         with self._connect() as conn:
542             row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
543             (job_id,)).fetchone()
544             if not row:
545                 return None
546             d = dict(row)
547             d["result"] = json.loads(d["result"]) if d["result"] else None
548             d["player_pool"] = json.loads(d["player_pool"]) if d["player_pool"] else
549             None
550             d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
551             return d
552
553     # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
554     -----
555     def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
556         """Attach/replace a job's JSON ExecutionPolicy."""
557         return self._execute_write(
558             "UPDATE jobs SET policy=? WHERE id = ?",
559             (json.dumps(policy) if policy is not None else None, job_id),
560             f"Failed to set policy for job {job_id}",
561         )
562
563     def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] =
564     None) -> bool:
565         """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
566         releasing the
567         worker. `claim_due_job` re-picks it when due ? cooperative, no master.
568         ``next_poll_at`` is
569         computed in SQLite's own UTC format so it compares directly to
570         CURRENT_TIMESTAMP."""
571         return self._execute_write(
572             "UPDATE jobs SET status='waiting', "
573             "next_poll_at=datetime('now', ?), "
574             "progress=COALESCE(?, progress) WHERE id = ?",

```

```

567         (f"+{max(0, int(delay_sec))} seconds", progress, job_id),
568         f"Failed to set job {job_id} waiting",
569     )
570
571 def seconds_until_next_due(self) -> Optional[float]:
572     """Seconds until the soonest parked ('waiting') job becomes due, so the worker
can
573     schedule a wake-up (EXECUTION_POLICY.md P3 self-scheduling). Returns 0.0 if one
is
574     already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
(0.0)."""
575     try:
576         with self._connect() as conn:
577             row = conn.cursor().execute(
578                 "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
579                 "    (julianday(next_poll_at) - julianday('now')) * 86400.0 END) AS
secs "
580                 "FROM jobs WHERE status='waiting']").fetchone()
581         if row is None or row["secs"] is None:
582             return None
583         return max(0.0, float(row["secs"]))
584     except Exception as e:
585         logger.error(f"Failed to compute next-due delay: {e}")
586         return None
587
588 def claim_due_job(self) -> Optional[Dict[str, Any]]:
589     """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
``next_poll_at`` is due ?
590     flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
591     double-claim (the DB row-claim is the only coordination needed; no master).
Earliest-due
592     first. Returns the claimed job dict, or None if nothing is runnable."""
593     try:
594         with self._connect() as conn:
595             cur = conn.cursor()
596             row = cur.execute(
597                 "SELECT id, status FROM jobs WHERE status='queued' "
598                 "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
599                 "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT
1").fetchone()
600             if not row:
601                 return None
602             cur.execute(
603                 "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
604                 "WHERE id=? AND status=?", (row["id"], row["status"]))
605             conn.commit()
606             if cur.rowcount != 1:
607                 return None # lost the race to a concurrent
worker
608             return self.get_job(row["id"])
609     except Exception as e:
610         logger.error(f"Failed to claim a due job: {e}")
611         return None
612
613 def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
614     """Crash recovery: any job still queued/running from a previous process is dead
615     (the executor is in-process). Mark failed so clients see a terminal state, not a
616     hang. Called once at startup. Returns the number of jobs swept."""
617     try:
618         with self._connect() as conn:
619             cur = conn.cursor()
620             cur.execute(
621                 "UPDATE jobs SET status='failed', error=?",

```

```

finished_at=CURRENT_TIMESTAMP "
622         "WHERE status IN ('queued', 'running')", (reason,))
623         conn.commit()
624         return cur.rowcount
625     except Exception as e:
626         logger.error(f"Failed to sweep stale jobs: {e}")
627         return 0
628
629     # --- Per-user model store (PERSONALIZATION_PLAN.md §2, v5) -----
630     # Groundwork ONLY: these persist/load the per-user model row. NOTHING on the served
631     # pipeline reads them yet ? the serving bridge (backend/pipeline/personalization/
632     # serving.py) resolves them, and wiring that bridge into the production decision
seam
633     # (`fusion_hybrid._select_for_handoff`) is the NEXT, owner-gated PR. NULL user_id =
634     # the local install. `fusion_config` / `classifier_json` / `promotion_evidence` are
635     # JSON-serialised dicts; the rest are scalar columns.
636
637     def upsert_user_model(self, user_id: Optional[str], *, version: int = 1,
638                           baseline_version: Optional[str] = None,
639                           feature_schema_hash: Optional[str] = None,
640                           fusion_config: Optional[Dict[str, Any]] = None,
641                           classifier_json: Optional[Dict[str, Any]] = None,
642                           promotion_evidence: Optional[Dict[str, Any]] = None,
643                           n_videos_at_fit: int = 0,
644                           is_active: bool = False) -> Optional[int]:
645         """Insert or replace ONE per-user model row keyed by `(user_id, version)``.
646
647         Idempotent on that key (UPSERT, not INSERT-OR-REPLACE: REPLACE would re-allocate
the
648         rowid). When `is_active` is True this row becomes the user's single active
model ?
649         any previously-active row for the SAME user is first cleared, so the
650         `idx_user_models_one_active` partial-unique invariant always holds (one active
model
651         per user). Returns the row id, or None on failure.
652         """
653         try:
654             with self._connect() as conn:
655                 cur = conn.cursor()
656                 if is_active:
657                     # Demote the current active row first (NULL user_id = local: IS NULL
match).
658
659                     if user_id is None:
660                         cur.execute(
661                             "UPDATE user_models SET is_active = 0 "
662                             "WHERE user_id IS NULL AND is_active = 1 AND version != ?",
663                             (version,))
664                     else:
665                         cur.execute(
666                             "UPDATE user_models SET is_active = 0 "
667                             "WHERE user_id = ? AND is_active = 1 AND version != ?",
668                             (user_id, version))
669                 cur.execute(
670                     """INSERT INTO user_models
671                        (user_id, version, baseline_version, feature_schema_hash,
672                        fusion_config, classifier_json, promotion_evidence,
673                        n_videos_at_fit, is_active)
674                        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
675                        ON CONFLICT(user_id, version) DO UPDATE SET
676                        baseline_version=excluded.baseline_version,
677                        feature_schema_hash=excluded.feature_schema_hash,
678                        fusion_config=excluded.fusion_config,
679                        classifier_json=excluded.classifier_json,
680                        promotion_evidence=excluded.promotion_evidence,
681                        n_videos_at_fit=excluded.n_videos_at_fit,

```

```

681             is_active=excluded.is_active"",
682             (user_id, version, baseline_version, feature_schema_hash,
683             json.dumps(fusion_config) if fusion_config is not None else None,
684             json.dumps(classifier_json) if classifier_json is not None else
None,
685             json.dumps(promotion_evidence) if promotion_evidence is not None
else None,
686             n_videos_at_fit, 1 if is_active else 0))
687         conn.commit()
688         rid = cur.execute(
689             "SELECT id FROM user_models WHERE "
690             + ("user_id IS NULL" if user_id is None else "user_id = ?")
691             + " AND version = ?",
692             ((version,) if user_id is None else (user_id, version))).fetchone()
693         return int(rid[0]) if rid else None
694     except Exception as e:
695         logger.error(f"Failed to upsert user_model (user={user_id}, v={version}):
{e}")
696         return None
697
698     def get_active_user_model(self, user_id: Optional[str]) -> Optional[Dict[str, Any]]:
699         """Return the single active per-user model row for ``user_id`` (NULL = local
install),
700         with ``fusion_config`` / ``classifier_json`` / ``promotion_evidence`` JSON
deserialised,
701         or None if the user has no active model. Does not touch the served pipeline."""
702         with self._connect() as conn:
703             if user_id is None:
704                 row = conn.cursor().execute(
705                     "SELECT * FROM user_models WHERE user_id IS NULL AND is_active =
1").fetchone()
706             else:
707                 row = conn.cursor().execute(
708                     "SELECT * FROM user_models WHERE user_id = ? AND is_active = 1",
709                     (user_id,)).fetchone()
710             if not row:
711                 return None
712             d = dict(row)
713             for col in ("fusion_config", "classifier_json", "promotion_evidence"):
714                 d[col] = json.loads(d[col]) if d[col] else None
715             return d
716

```

7. assistant (2026-06-21T02:34:31.416Z)

Now let me search for any additional components related to API routes and cost tracking.

8. user (2026-06-21T02:34:31.721Z)

```

backend\eval\__init__.py
backend\eval\classifier.py
backend\eval\training.py
backend\providers\__init__.py
backend\sports\__init__.py
backend\sports\badminton.py
backend\sports\base.py
backend\utils\__init__.py
backend\utils\geometry.py
backend\annotations\base.py
backend\annotations\file_provider.py
backend\annotations\automated_provider.py
backend\annotations\__init__.py
backend\tools\rally_slicer.py
backend\tools\__init__.py
backend\eval\gemini_refine.py

```

backend\eval\audio__init__.py
backend\eval\audio\e1_probe.py
backend\eval\export_wasb_segments.py
backend\pipeline\audio__init__.py
backend\pipeline\validators__init__.py
backend\eval\metrics.py
backend\eval\regression.py
backend\eval\annotations.py
backend\features__init__.py
backend\reporting\harvest.py
backend\reporting\run_signals.py
backend\eval\calibrate_wasb.py
backend\eval\calibration.py
backend\eval\eval_partial_labels.py
backend\eval\rally_seq_proto.py
backend\features\rally_seq.py
backend\pipeline\detectors__init__.py
backend\pipeline\segmenters\base.py
backend\pipeline\validators\blur.py
backend\pipeline\validators\format.py
backend\pipeline\validators\pts_continuity.py
backend\pipeline\validators\relevance.py
backend\pipeline\validators\resolution_fps.py
backend\pipeline\validators\scene_cut.py
backend\providers\base.py
backend\reporting__init__.py
backend\eval\calibrate_local.py
backend\eval\gt_loader.py
backend\eval\harness.py
backend\pipeline\audio\hit_detector.py
backend\pipeline\validators\base.py
backend\utils\paths.py
backend\utils\video.py
backend\pipeline\segmenters\tracknet_hybrid.py
backend\pipeline\detectors\fusion_features.py
backend\pipeline\detectors\rally_gate.py
backend\pipeline\segmenters__init__.py
backend\tools\export.py
backend\utils\logging_setup.py
backend\pipeline\detectors\audio_features.py
backend\eval\eval_stats.py
backend\eval\score_calibration.py
backend\pipeline\detectors\person_detector.py
backend\infrastructure\execution_policy.py
backend\utils\run_telemetry.py
backend\eval\batch.py
backend\utils\hashing.py
backend\eval\fusion_golden.py
backend\eval\per_user_gate.py
backend\eval\per_user_eval.py
backend\eval\per_user_regression.py
backend\pipeline\personalization\serving.py
backend\pipeline\personalization__init__.py
backend\eval\served_gate_a.py
backend\eval\distill_local.py
backend\eval\promotion.py
backend\eval\serve_contrast.py
backend\eval\splits.py
backend\pipeline\detectors\base.py
backend\pipeline\detectors\wasb_runner.py
backend\eval\segmentation_metrics.py
backend\eval\tolerance_metrics.py
backend\eval>windowing.py
backend\results.py
backend\api__init__.py

```
backend\api\uploads.py
backend\config\loader.py
backend\eval\ablation_pdf.py
backend\pipeline\detectors\tracknet_runner.py
backend\api\job_worker.py
backend\eval\ablation.py
backend\eval\experiment.py
backend\eval\fusion_audio.py
backend\eval\fusion_compare.py
backend\eval\golden_fixtures.py
backend\eval\golden_real_fixtures.py
backend\infrastructure\database.py
backend\pipeline\segmenters\motion.py
backend\providers\gemini.py
backend\pipeline\detectors\court_gate.py
backend\eval\rally_seg_eval.py
backend\config\__init__.py
backend\pipeline\detectors\stub_runner.py
backend\pipeline\segmenters\_keyhint.py
(Results are truncated. Consider using a more specific path or pattern.)
```

9. assistant (2026-06-21T02:34:33.858Z)

Let me look at the config files and check for any cost/billing related code:

10. user (2026-06-21T02:34:34.843Z)

EISDIR: illegal operation on a directory, read 'C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config'

11. user (2026-06-21T02:34:35.143Z)

```
Found 11 files
backend\pipeline\detectors\wasb_infer.py
backend\eval\rally_seg_eval.py
backend\eval\golden_real_fixtures.py
backend\eval\golden_fixtures.py
backend\eval\tolerance_metrics.py
backend\pipeline\detectors\person_detector.py
backend\assets\fonts\LICENSE-Apache-2.0.txt
backend\eval\calibrate_local.py
backend\eval\harness.py
backend\annotations\file_provider.py
backend\eval\__init__.py
```

12. user (2026-06-21T02:34:37.111Z)

```
1      """Configuration package.
2
3      Public API:
4          from backend.config import load_config, AppConfig, Config
5
6      This package is the canonical place for all configuration access.
7      """
8
9      from .loader import load_config, save_default_config
10     from .models import ( # re-export common ones
11         AppConfig,
12         Config,
13         IndexingConfig,
14         StitchingConfig,
15         StorageConfig,
16         ValidationConfig,
17     )
```



```

18
19     __all__ = [
20         "load_config",
21         "save_default_config",
22         "AppConfig",
23         "Config",
24         "IndexingConfig",
25         "StitchingConfig",
26         "StorageConfig",
27         "ValidationConfig",
28     ]
29

```

13. user (2026-06-21T02:34:37.122Z)

backend\config\loader.py
 backend\config__init__.py
 backend\config\models.py

14. user (2026-06-21T02:34:37.162Z)

```

1     """Configuration loader and saver.
2
3     Provides a single place to load validated AppConfig with built-in defaults
4     merged with optional file overrides (config.json).
5
6     This replaces scattered json.load + .get() patterns throughout the codebase.
7     """
8
9     from __future__ import annotations
10
11     import json
12     import logging
13     from pathlib import Path
14     from typing import Any
15
16     from pydantic import BaseModel
17
18     from .models import AppConfig
19
20     logger = logging.getLogger(__name__)
21
22     DEFAULT_CONFIG_PATH = Path("config.json")
23
24
25     def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
26     "") -> list[str]:
27         """Dotted paths in `data` that the typed config will not honor.
28
29         Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
30         kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
31         typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
32         user's intent is lost ? collect every such key so load_config can warn.
33         """
34         unknown: list[str] = []
35         fields = model_cls.model_fields
36         for key, val in data.items():
37             if key not in fields:
38                 unknown.append(prefix + key)
39                 continue
40             ann = fields[key].annotation
41             if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
42             BaseModel):
43                 unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}.")

```

```

43
44
45 def _get_builtin_defaults() -> dict[str, Any]:
46     """Return a plain dict of the current built-in defaults.
47
48     The AppConfig model is the single source of truth for defaults.
49     """
50     return AppConfig().model_dump(mode="json")
51
52
53 def load_config(path: str | Path | None = None) -> AppConfig:
54     """Load configuration with the following precedence:
55
56     1. Built-in defaults defined in the Pydantic models.
57     2. Values from the JSON file (if present and readable).
58     3. (Future) environment / CLI overrides.
59
60     Missing file or unreadable file ? returns a fully defaulted AppConfig.
61     Unknown keys in the file are tolerated (extra="allow" on the model).
62     """
63     cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
64
65     file_data: dict[str, Any] = {}
66     if cfg_path.exists():
67         try:
68             with open(cfg_path, "r", encoding="utf-8") as f:
69                 file_data = json.load(f)
70         except Exception as exc:
71             # Non-fatal: continue with defaults + whatever we could parse.
72             # In production we might log here.
73             logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")
74
75     # A config.json that is valid JSON but not an OBJECT (e.g. `[ ]`, `42`, `"x"`,
`null`) would
76     # otherwise crash startup: a truthy non-mapping hits `.items()` in
`_unknown_key_paths`, and any
77     # non-mapping breaks the `{**defaults, **file_data}` unpack. Degrade to defaults +
warn, per the
78     # loader's "bad input is non-fatal" contract.
79     if not isinstance(file_data, dict):
80         logger.warning("%s is not a JSON object (got %s); ignoring it and using
defaults.",
81                        cfg_path, type(file_data).__name__)
82         file_data = {}
83
84     if file_data:
85         unknown = _unknown_key_paths(file_data, AppConfig)
86         if unknown:
87             logger.warning(
88                 "[CONFIG WARNING] %s contains keys the typed config does not "
89                 "recognize (typo'd or removed knobs ? top-level extras are kept "
90                 "but unread; unknown section keys are silently dropped): %s",
91                 cfg_path, ", ".join(sorted(unknown)),
92             )
93
94     # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
95     merged = {**_get_builtin_defaults(), **file_data}
96
97     return AppConfig.model_validate(merged)
98
99
100 def save_default_config(path: str | Path | None = None) -> Path:
101     """Write a fresh default config.json.
102

```

```

103     Used by scripts/doctor.py diagnostics and the --setup CLI path.
104     Overwrites if the file exists.
105     """
106     cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
107     cfg = AppConfig()
108     cfg_path.parent.mkdir(parents=True, exist_ok=True)
109     cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
110     return cfg_path
111
112
113     # Backwards-compatible alias used by some older call sites during transition.
114     def get_default_config() -> dict[str, Any]:
115         """Return defaults as a plain dict (for transitional use only)."""
116         return _get_builtin_defaults()
117

```

15. user (2026-06-21T02:34:39.456Z)

```

1     """Pydantic models for the application configuration.
2
3     This module defines the canonical, validated, typed configuration for the
4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6     The models support graceful migration from old config.json files via extra="allow".
7     """
8
9     from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         relevance: ValidationRelevanceConfig =
31         Field(default_factory=ValidationRelevanceConfig)
32
33     class TrackNetConfig(BaseModel):
34         wsl_distro: str = "Ubuntu"
35         repo_dir: str = "~/models/TrackNetV4"
36         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37         conda_env: str = "TrackNetV4"
38         weights_path: str = ""
39         queue_length: int = 5
40         stage_in_wsl: bool = True
41         wsl_stage_dir: str = "~/clips"
42         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43         velocity_threshold: float = 0.02
44         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45         # decoded frame cache) are deleted automatically ? they are pure intermediates,
46         # regenerable from the source, and the dominant disk consumer (~GBs/video). Set

```

```

47         # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48         keep_frames: bool = False
49         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50         # 0 = disabled.
51         min_free_gb: float = 0.0
52
53
54     class WasbIndexConfig(BaseModel):
55         """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58         these knobs actually take effect ? previously this whole block was silently dropped
59         because nested config sections defaulted to extra='ignore'.
60         """
61         wsl_distro: str = "Ubuntu"
62         repo_dir: str = "~/models/WASB-SBDT"
63         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64         conda_env: str = "wasb"
65         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66         sport: str = "badminton"
67         wsl_stage_dir: str = "~/clips"
68         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69         velocity_threshold: float = 0.02
70         # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
71         ---
72         # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
73         device: str = "cuda" # torch device for the native runner: cuda | mps | cpu
74         python_bin: str = "python" # the `wasb` conda env python (invoked by path, no
75         `conda activate`)
76         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
77         # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
78         regenerable
79         # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
80         resumes.
81         keep_frames: bool = False
82         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
83         disabled.
84         min_free_gb: float = 0.0
85         # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
86         # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
87         for a
88         # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
89         to the
90         # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
91         WSL disk,
92         # 1194/1195 frames byte-identical; native run-to-run byte-identical /
93         deterministic).
94         # Tri-state:
95         # None = per-runner default ? the WSL runner keeps DISK extraction
96         (unchanged Windows
97         # behaviour); the Linux-native runner (GPU box) defaults to STREAMING
98         (the perf win).
99         # True/False = force on/off for BOTH runners (e.g. set False for a container cv2
100         can't seek).
101         stream_video: Optional[bool] = None
102
103
104     class FusionConfig(BaseModel):
105         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
106         P2).
107
108         Every default reproduces control behaviour: the fusion segmenter persists the
109         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
110         is

```

```

97     true ? the person-cue features, but it does NOT gate the served handoff unless a
98     threshold is raised (`min_crossings`/`min_players` default 0 = OFF). The court
mask
99     is optional: `court_polygon=None` ? Gate B counts every detection (player-count-
only);
100     supplied ? off-court persons (spectators / adjacent courts) are excluded.
101     ""
102     # Which trajectory substrate seeds the windows (shuttle stays primary).
103     substrate: Literal["wasb", "tracknet"] = "wasb"
104     # Windowing velocity threshold for the fusion family (the engine reads
105     # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
106     # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
107     velocity_threshold: float = 0.02
108     # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
109     # enabled ? so the default path never imports torch / touches the GPU.
110     cue_flags: dict[str, bool] = Field(default_factory=dict)
111     # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
112     # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
113     min_crossings: int = Field(default=0, ge=0)
114     # Served Gate B: when the person cue is on, drop windows with median in-court
players
115     # < this. 0 = OFF.
116     min_players: int = Field(default=0, ge=0)
117     # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
118     court_polygon: Optional[list[list[float]]] = None
119     # Net line for the person two-sided-motion split (person features only ? the shuttle
120     # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
121     net_axis: Literal["x", "y"] = "y"
122     net_position: float = Field(default=0.5, ge=0.0, le=1.0)
123
124
125     class IndexingConfig(BaseModel):
126         default_segmenter: str = "yolo_hybrid"
127         use_gpu: bool = True
128         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
129         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
130         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
131         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
132         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
133         detector_impl: str = "auto"
134         motion_threshold: float = 0.08
135         min_segment_duration: float = 3.0
136         dead_time_merge_gap: float = 2.0
137         chunk_duration: float = 90.0
138         chunk_overlap: float = 10.0
139         detector_model: str = "fasterrcnn_resnet50_fpn"
140         detector_score_threshold: float = 0.5
141         yolo_velocity_threshold: float = 0.15
142         yolo_frame_skip: int = 3
143         yolo_tracker: str = "bytetrack.yaml"
144         yolo_prefetch_workers: int = 2
145         min_action_window_duration: float = 2.0
146         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
=a window
147         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
148         # `window_min_split_gap` s ? the most likely rally boundary), so one window can't
swallow
149         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
150         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13

```

```

151      # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
152      # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
153      max_window_duration: float = 6.0
154      window_min_split_gap: float = 0.5 # min internal gap (s) that qualifies as a split
point
155      # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
156      # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
157      # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
158      # Only cuts (recall can't drop). OFF by default until measured/promoted.
159      window_hard_cap: bool = False
160      # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
161      # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
162      # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
163      # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
164      # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
165      # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
166      # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
167      window_inactivity_end: float = 1.5
168      inactivity_rally_thresh: float = 0.20
169      inactivity_min_density: float = 0.30
170      # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
171      # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
172      # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
173      # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
174      # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
175      # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
176      # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
177      # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
178      window_blob_fallback: bool = False
179      # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
180      # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
181      # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
182      # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
183      window_blob_factor: float = 4.0
184      log_yolo_candidates: bool = True
185      store_yolo_candidates: bool = True
186      ai_handoff_padding: float = 2.0
187      ai_max_workers: int = 5
188      # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
189      # chunks of a high-bitrate (4K) source blow past the provider upload cap
190      # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
191      # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate

```

```

192     # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
193     # Matches gemini_refine's --clip-scale-width default.
194     ai_chunk_scale_width: int = 1280
195     # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
196     # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
197     # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
198     # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
199     # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
200     skip_ai_handoff: bool = False

```

16. user (2026-06-21T02:34:41.859Z)

```

200     skip_ai_handoff: bool = False
201     # Optional debug knob: trim every video to the first N seconds before processing.
202     debug_duration: Optional[float] = None
203
204     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
205     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
206     fusion: FusionConfig = Field(default_factory=FusionConfig)
207
208     @field_validator("default_segmenter")
209     @classmethod
210     def segmenter_must_be_registered(cls, v: str) -> str:
211         # Registry check happens at runtime in main/segmenter selection.
212         # Here we just allow any string for forward compatibility.
213         return v
214
215     @model_validator(mode="after")
216     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
217         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
218         # step makes `while t < duration: t += step` loop forever, growing memory before
any
219         # GPU work. Reject the bad config at load time instead.
220         if self.chunk_overlap >= self.chunk_duration:
221             raise ValueError(
222                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
223                 f"({self.chunk_duration}); otherwise chunking never advances."
224             )
225         return self
226
227
228     class OutputConfig(BaseModel):
229         default_highlights_name: str = "highlights.mp4"
230         db_name: str = "sports_indexer.db"
231         # Directory where the DB, highlight reels, and processed clips are written.
232         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
233         output_dir: str = "output"
234
235
236     class WatermarkConfig(BaseModel):
237         """Branding overlay burned into each highlight clip during the per-clip re-encode
238         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
239         (under `stitching.watermark` in config.json) to turn it off. The text is fully
240         configurable. The concat stage stays stream-copy (-c copy)."""
241         enabled: bool = True
242         text: str = "Generated by Khelsutra.guru"
243         logo_path: str = "" # optional transparent PNG overlay; "" = no logo
244         font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family
below

```

```

245     font: str = "Sans"           # fontconfig family used when font_path is empty
246     font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
fraction of frame height
247     opacity: float = Field(default=0.85, ge=0.0, le=1.0)
248     box: bool = True             # faint backing box behind the text for legibility
249     margin: int = Field(default=24, ge=0) # px inset from the chosen corner
250     position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"
251
252
253     class StitchingConfig(BaseModel):
254         begin_padding: float = 0.5
255         end_padding: float = 0.5
256         use_cache: bool = False
257         min_shot_count: int = 1
258         # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
259         # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
260         # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
261         # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
262         # local detector over-fires and its false positives are mostly SHORT (motion blips,
263         # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
264         # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
265         min_rally_duration: float = Field(default=0.0, ge=0.0)
266         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
267
268
269     class LoggingConfig(BaseModel):
270         """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
271         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
272         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
273         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
274         # them to WARNING; set true to restore full chatter for request-flow debugging.
275         verbose_http: bool = False
276
277
278     class SecurityConfig(BaseModel):
279         # When set, /api/process_video_path must resolve under this directory (hosted/tenant
280         # mode). Default None preserves the single-user local 'any local file' workflow.
281         allowed_video_root: Optional[str] = None
282
283         # --- Chunked upload (B2, PR-2) ---
284         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
285         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
286         # contain upload_dir or /api/process will reject the uploaded paths.
287         upload_dir: str = "output/uploads"
288         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
289         # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
290         max_upload_mb: int = 4096
291         max_part_mb: int = 95
292         allowed_upload_extensions: List[str] = ["mp4", "mov"]
293
294
295     class StorageConfig(BaseModel):
296         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
297         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
298         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
299         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?

```



```

300     **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
301     (boto3/google auth chains supply credentials). See ``backend/storage/``.
302     backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
303     # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
304     output_root: str = ""
305     # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
306     # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
307     # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
308     # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
309     # the legacy flat layout is used unchanged.
310     single_folder_per_video: bool = False
311     # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
312     workspace_root: str = ""
313     # Cloud namespace prefix under the root so per-video folders sit tidily beside
314     # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
315     workspace_prefix: str = "workspaces"
316     local: dict[str, Any] = Field(default_factory=dict)
317     s3: dict[str, Any] = Field(default_factory=dict)
318     gcs: dict[str, Any] = Field(default_factory=dict)
319     gdrive: dict[str, Any] = Field(default_factory=dict)
320
321
322     class AppConfig(BaseModel):
323         """Root configuration object.
324
325         All runtime configuration should be accessed via an instance of this class
326         (or the loader) rather than raw dict lookups.
327         """
328
329         sport: Sport = "badminton"
330         ai_provider: str = "gemini"
331         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
332         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
333         ai_model: str = "gemini-flash-latest"
334
335         validation: ValidationConfig = Field(default_factory=ValidationConfig)
336         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
337         output: OutputConfig = Field(default_factory=OutputConfig)
338         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
339         security: SecurityConfig = Field(default_factory=SecurityConfig)
340         logging: LoggingConfig = Field(default_factory=LoggingConfig)
341         storage: StorageConfig = Field(default_factory=StorageConfig)
342
343         # Allow extra keys during transition so existing config.json files do not break.
344         model_config = {
345             "extra": "allow",
346             "validate_assignment": True,
347         }
348
349         # Temporary migration shim for legacy dict-style access (config.get("section",
{})).get(...)).
350         # Will be removed once all call sites (validators, segmenters, etc.) move to typed
351         # attribute access / the typed config. A nested *section* is returned as a plain
dict
352         # via model_dump, so legacy chained access keeps working ? returning the Pydantic
353         # section model instead breaks it (a model has no .get), which bit every validator
on

```

```

354     # real runs. Scalar leaves are returned as-is.
355     def get(self, key: str, default: Any = None) -> Any:
356         _MISSING = object()
357         val: Any = _MISSING
358         if key in type(self).model_fields or hasattr(self, key):
359             val = getattr(self, key, _MISSING)
360         if val is _MISSING:
361             # Legacy convenience: find a leaf key inside any nested section. Sections
are
362             # discovered dynamically from the model fields, so a future top-level
section
363             # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
364             for fname in type(self).model_fields:
365                 section = getattr(self, fname, None)
366                 if isinstance(section, BaseModel) and hasattr(section, key):
367                     val = getattr(section, key)
368                     break
369             if val is _MISSING:
370                 return default
371             if isinstance(val, BaseModel):
372                 return val.model_dump(mode="json")
373             return val
374
375     def __getitem__(self, key: str) -> Any:
376         val = self.get(key)
377         if val is None and key not in type(self).model_fields:
378             # Re-raise like a real dict for unknown top-level
379             raise KeyError(key)
380         return val
381
382
383     # Convenience type alias for callers that want to annotate
384     Config = AppConfig
385

```

17. assistant (2026-06-21T02:34:43.934Z)

Now let me check what's in the API directory and look for any compute/dispatch related files:

18. user (2026-06-21T02:34:44.526Z)

```

C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api\job_worker.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api\uploads.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api\__init__.py

```

19. user (2026-06-21T02:34:46.310Z)

```

1     """HTTP API surface for the indexer (FastAPI routers)."""
2

```

20. assistant (2026-06-21T02:34:48.528Z)

Let me check the infrastructure directory for any cost/job-related files:

21. user (2026-06-21T02:34:48.792Z)

```

C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\execution_policy.py

```

22. user (2026-06-21T02:34:50.497Z)

```

1     """Per-job execution policy: bounded, observable waits for long/flaky ops
(EXECUTION_POLICY.md P1).

```

```

2
3     Turns an unbounded blocking call (the GX010125 Gemini hang) into a bounded one:
4     - ``run_bounded`` ? a HARD per-attempt timeout (``op_timeout_sec``, the hang-killer) +
retry-with-backoff
5         for FAILURES (an exception) *and* HANGS (no return in time). A hung attempt's worker
thread is a daemon,
6         so it is abandoned, not joined ? it never blocks the process.
7     - ``poll_until`` ? a bounded poll loop for an in-progress external op
(``poll_every_n_sec`` cadence,
8         ``max_wait_sec`` deadline).
9
10    **Testability:** ``sleep``/``monotonic`` are injected, so the whole module is unit-
tested with zero real
11    waiting (a fake clock advances instantly).
12
13    **Logging discipline** (so polling is VISIBLE but never NOISY): every poll/retry logs at
**DEBUG**; state
14    transitions (start, resolved, hung, gave-up) log at **INFO/WARNING**. INFO stays clean;
enable DEBUG on the
15    ``execution.policy`` logger to watch the poll trail.
16    ""
17    from __future__ import annotations
18
19    import logging
20    import threading
21    import time as _time
22    from dataclasses import asdict, dataclass
23    from enum import Enum
24    from typing import Callable, Optional, TypeVar
25
26    T = TypeVar("T")
27
28    #: Dedicated logger so callers can dial polling visibility independently of the app's
root level.
29    LOG = logging.getLogger("execution.policy")
30
31
32    class WaitMode(str, Enum):
33        """How a job treats a long wait. ``str``-valued so it serialises straight to JSON on
the job row."""
34        WAIT_AND_RESUME = "wait_and_resume" # default: bounded wait; over budget ->
reschedule/partial (P2/P3)
35        ABORT = "abort" # one attempt; on hang/fail, stop immediately
36        BLOCK = "block" # bounded in-process blocking wait (no
reschedule)
37
38
39    class PolicyTimeout(TimeoutError):
40        """An op hung past ``op_timeout_sec`` or never became ready within
``max_wait_sec``."""
41
42
43    @dataclass(frozen=True)
44    class ExecutionPolicy:
45        """Declarative resilience knobs, attachable per job/op (JSON-serialisable; see
EXECUTION_POLICY.md)."""
46        mode: WaitMode = WaitMode.WAIT_AND_RESUME
47        poll_every_n_sec: float = 10.0
48        op_timeout_sec: float = 120.0 # HARD per-attempt deadline ? the hang-killer
that was missing
49        max_wait_sec: float = 1800.0 # total wall budget for a poll loop
50        max_retries: int = 5 # retries (=> max_retries+1 attempts) for
failures/hangs
51        backoff_base_sec: float = 3.0 # exponential: backoff_base * 2**attempt
52        on_timeout: str = "reschedule" # reschedule | partial | fail (consumed by

```

```

P2/P3)
53         resumable: bool = True
54
55     def to_dict(self) -> dict:
56         d = asdict(self)
57         d["mode"] = self.mode.value
58         return d
59
60     @classmethod
61     def from_dict(cls, d: dict) -> "ExecutionPolicy":
62         known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
63         kw = {k: v for k, v in d.items() if k in known}
64         if "mode" in kw and not isinstance(kw["mode"], WaitMode):
65             kw["mode"] = WaitMode(kw["mode"])
66         return cls(**kw)
67
68
69     #: The sane default (also what a job gets if it declares no policy).
70     DEFAULT_POLICY = ExecutionPolicy()
71
72
73     def _backoff_sec(policy: ExecutionPolicy, attempt: int) -> float:
74         """Deterministic exponential backoff (no jitter ? reproducible tests)."""
75         return policy.backoff_base_sec * (2 ** attempt)
76
77
78     def run_bounded(fn: Callable[[], T], policy: ExecutionPolicy = DEFAULT_POLICY, *,
79                    label: str = "op", logger: Optional[logging.Logger] = None,
80                    sleep: Callable[[float], None] = _time.sleep) -> T:
81         """Run ``fn()`` under a hard per-attempt timeout, retrying FAILURES and HANGS with
82         backoff.
83
84         Returns ``fn``'s value on success. Raises ``PolicyTimeout`` if the final attempt
85         hung, or re-raises the
86         last exception if the final attempt failed. ``mode=ABORT`` => a single attempt (no
87         retries).
88
89         """
90         log = logger or LOG
91         attempts = 1 if policy.mode is WaitMode.ABORT else policy.max_retries + 1
92         last_exc: Optional[BaseException] = None
93         for attempt in range(attempts):
94             box: dict = {}
95             done = threading.Event()
96
97             # box/done bound as defaults (not closed over) ? the thread is started + joined
98             # within this
99             # same iteration, but explicit binding keeps each attempt isolated and silences
100             B023.
101
102             def _runner(_box: dict = box, _done: threading.Event = done) -> None:
103                 try:
104                     _box["val"] = fn()
105                 except BaseException as e: # relay ANY error (incl. timeouts) to the
106                     # caller thread
107                     _box["exc"] = e
108                 finally:
109                     _done.set()
110
111             threading.Thread(target=_runner, name=f"bounded:{label}", daemon=True).start()
112             if done.wait(timeout=policy.op_timeout_sec):
113                 if "exc" in box:
114                     last_exc = box["exc"]
115                     log.info("%s: attempt %d/%d failed: %s", label, attempt + 1, attempts,
116                             last_exc)
117                 else:
118                     if attempt:

```

```

110             log.info("%s: succeeded on attempt %d/%d", label, attempt + 1,
attempts)
111             return box["val"] # type: ignore[return-value]
112         else:
113             last_exc = PolicyTimeout(f"{label} hung >
op_timeout_sec={policy.op_timeout_sec}s")
114             log.warning("%s: attempt %d/%d HUNG (>%.0fs) ? abandoning", label, attempt +
1, attempts,
115                         policy.op_timeout_sec)
116             if attempt < attempts - 1:
117                 wait = _backoff_sec(policy, attempt)
118                 log.debug("%s: backing off %.1fs before retry", label, wait)
119                 sleep(wait)
120             assert last_exc is not None
121             log.warning("%s: gave up after %d attempt(s): %s", label, attempts, last_exc)
122             raise last_exc
123
124
125     def poll_until(check: Callable[[], Optional[T]], policy: ExecutionPolicy =
DEFAULT_POLICY, *,
126                  label: str = "op", logger: Optional[logging.Logger] = None,
127                  sleep: Callable[[float], None] = _time.sleep,
128                  monotonic: Callable[[], float] = _time.monotonic) -> T:
129         """Poll ``check()`` every ``poll_every_n_sec`` until it returns a non-``None``
value, or raise
130         ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
still in progress.
131
132         Polls log at DEBUG (no INFO spam); the wait start + resolution + give-up log at
INFO/WARNING.
133         """
134         log = logger or LOG
135         start = monotonic()
136         polls = 0
137         log.info("%s: waiting (poll_every=%.0fs, max_wait=%.0fs)",
138                 label, policy.poll_every_n_sec, policy.max_wait_sec)
139         while True:
140             result = check()
141             if result is not None:
142                 log.info("%s: ready after %d poll(s) / %.0fs", label, polls, monotonic() -
start)
143                 return result
144             elapsed = monotonic() - start
145             if elapsed >= policy.max_wait_sec:
146                 log.warning("%s: still not ready after %.0fs (max_wait) ? giving up", label,
elapsed)
147                 raise PolicyTimeout(f"{label} not ready within
max_wait_sec={policy.max_wait_sec}s")
148             polls += 1
149             log.debug("%s: poll %d, %.0fs elapsed, not ready", label, polls, elapsed)
150             sleep(policy.poll_every_n_sec)
151

```

23. assistant (2026-06-21T02:34:52.754Z)

Now let me check for any references to Cloud Run, GPU, or dispatching:

24. user (2026-06-21T02:34:54.759Z)

```

C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/config/models.py:    timeout_sec: int
= 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/config/models.py:    timeout_sec: int
= 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)

```

```

C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/config/models.py:    # enabled ? so
the default path never imports torch / touches the GPU.
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/config/models.py:    use_gpu: bool =
True
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/config/models.py:    # GPU work.
Reject the bad config at load time instead.
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/ablation.py:Pure + offline + GPU-
free. Reuses the persisted golden feature substrate (``fusion_golden.py``
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/ablation.py:    Returns the
written path. ``reportlab`` is an offline, pure-Python dependency (no GPU)."""
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/batch.py:aggregation) so they can
be unit-tested without touching video files, the GPU,
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/experiment.py:(per-frame signals
? run once on the GPU/WSL box, persisted into
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/experiment.py:with no GPU and
zero production risk. So most experiments are pure offline
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/fusion_audio.py:Reuses the
person-feature JSONs from `fusion_golden.py` (no GPU re-detect): `augment` runs
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/fusion_audio.py:Run (no GPU):
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/fusion_compare.py:Run (no GPU):
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/fusion_golden.py:"""Offline
golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/fusion_golden.py:CPU offline. So
the GPU touches exactly one thing ? person detection ? and the lift is
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/fusion_golden.py:Run (on the GPU
box):
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/fusion_golden.py:`fusion_compare`
with different variants/thresholds forever without re-detecting on the GPU.
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/fusion_golden.py:    `detector`
(anything with ``detections_over_windows``) is injectable for GPU-free tests;
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/fusion_golden.py:    detector
= PersonDetector({"indexing": {"use_gpu": True}})
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/fusion_golden.py:    # forward ?
no per-window seek). Then compute features per window (pure, no GPU).
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/fusion_golden.py:    # Run-
telemetry black box: a long GPU run can be OS-killed with no traceback, so keep a tiny
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/fusion_golden.py:    ap =
argparse.ArgumentParser(description="Extract golden-set shuttle+person features (fusion P3 GPU
step).")
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/per_user_eval.py:Pure, offline,
deterministic given ``seed``; no GPU/torch/cv2; no file I/O (callers pass
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/per_user_regression.py:Pure-CPU,
deterministic given ``seed`` (the only randomness is the CI bootstrap); no GPU / torch /

```

C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/segmentation_metrics.py:so it runs in the GPU-free / numpy-free lanes too.
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/served_gate_a.py:GPU-free and Gemini-free: it stops at the handoff-selection boundary (the windows that WOULD
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/served_gate_a.py:the GPU ? it just re-scores cached WASB trajectories.
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/served_gate_a.py: person cue OFF (so no GPU / torch is touched).""
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/served_gate_a.py: ``trajectory\_to\_action\_windows`` operating point. No GPU, no Gemini: the person cue is OFF.
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/serve_contrast.py:Pure CPU, GPU-free, no Gemini, no DB. Reuses only shipped pieces (windowing, rally_gate,

25. assistant (2026-06-21T02:34:56.916Z)

Let me search for any documentation or planning files about hosting/cloud architecture:

26. user (2026-06-21T02:34:57.792Z)

C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/DEFERRED_HARDENING_PLAN-COMPLETED-2026-06-14.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/past_projects/low-regret-moves/LOW_REGRET_MOVES_PLAN.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/research/AUDIO_RALLY_DETECTION_PLAN.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/DEFERRED_HARDENING_PLAN.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/HOSTING_PLAN.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/I18N_PLAN.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/MULTI_SIGNAL_FUSION_PLAN.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/PERSONALIZATION_PLAN.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/UI_ALPHA_EXECUTION_PLAN.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/bhi-execpolicy/docs/archives/past_projects/low-regret-moves/LOW_REGRET_MOVES_PLAN.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/bhi-execpolicy/docs/archives/research/AUDIO_RALLY_DETECTION_PLAN.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/bhi-execpolicy/docs/DEFERRED_HARDENING_PLAN.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/bhi-execpolicy/docs/HOSTING_PLAN.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/bhi-execpolicy/docs/I18N_PLAN.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/bhi-execpolicy/docs/MULTI_SIGNAL_FUSION_PLAN.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/bhi-execpolicy/docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/bhi-execpolicy/docs/PERSONALIZATION_PLAN.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/bhi-execpolicy/docs/UI_ALPHA_EXECUTION_PLAN.md

C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/boxhill17/docs/archives/DEFERRED_HARDENING_PLAN-COMPLETED-2026-06-14.md

27. user (2026-06-21T02:35:00.044Z)

```
1      # Hosting Plan ? khelsutra.guru
2
3      **Status:** PROPOSAL ? awaiting owner ratification.
4      **Date:** 2026-06-10 . **Constraints:** budget $10?25/mo . GCP credits available .
DigitalOcean
5      acceptable . alpha backend = owner's GPU box behind a tunnel . 10?25 alpha testers .
registrar = GoDaddy.
6
7      > **? Update (2026-06-20, ADR-013): DigitalOcean is DROPPED for COMPUTE** ? GCP is the
sole compute
8      > stack (training + serving). The ***"All-DigitalOcean"*** compute option below is
retired; DO Spaces
9      > remains only a *possible* S3-compatible storage target. Edge/site hosting choices here
are unaffected.
10     **Companion:** `docs/UI_PROPOSAL.md`.
11
12     ## 1. Recommendation in one line
13
14     **Alpha: free edge + your box** (Cloudflare DNS/Pages/Access/Tunnel ? FastAPI on the GPU
box) at
15     **$0/mo**, saving GCP credits; **Beta: GCP control plane + your box as pull-worker**
(Cloud Run +
16     GCS + Firebase Auth, credits-covered), keeping the edge where it's free.
17
18     Rationale for the Cloudflare pieces despite the GCP preference: GCP has no free
equivalent of the
19     Tunnel + Access combo (IAP requires a load balancer, ~$18/mo standing cost), and the
alpha needs
20     exactly three things GCP charges for but Cloudflare gives free: a secure tunnel to a
home box with
21     a custom domain, email-allowlist auth (free ?50 users ? fits 10?25 testers), and static
hosting.
22     Credits are better spent at beta on compute/storage, which is where GCP is excellent.
23
24     ## 2. Alpha topology ($0/mo)
25
26     ```
27     tester browser
28     ? https
29     ?
30     Cloudflare edge (free): DNS + TLS + Access (email-OTP allowlist)
31     ??? app.khelsutra.guru ? Cloudflare Pages (React SPA; auto-deploy from GitHub)
32     ??? api.khelsutra.guru ? Cloudflare Tunnel (outbound-only `cloudflared` service on
the box)
33     ?
34     ?
35     FastAPI :8000 on the GPU box (Windows)
36     ? Gemini segmenter path (no GPU in request path)
37     ? async jobs (#16), SQLite WAL, output/ on disk
38     ```
39
40     - **No inbound ports opened** on the home network; `cloudflared` dials out. Home IP
never exposed.
41     - **Auth:** Cloudflare Access in front of BOTH hostnames; backend trusts the
42     `Cf-Access-Authenticated-User-Email` header (verify the Access JWT for rigor) for per-
user scoping.
43     - **Upload limit:** free-tier proxy caps request bodies at **100 MB** ? SPA uploads in
?90 MB
44     chunks (UI_PROPOSAL B2). Match videos (1?4 GB) flow fine as chunks.
```



```

45     - **Box offline = service offline.** Acceptable for alpha; SPA shows a friendly
"processing engine
46     offline" banner (a `/api/health` ping). Beta removes this with the queue.
47     - **Costs:** Pages, Tunnel, Access, DNS = $0. Only the Gemini per-run cost
(~$0.05/match) and
48     domain renewal.
49
50     ## 3. Beta topology (GCP credits; ~$15?20/mo post-credits)
51
52     ```
53     browser ??? Cloudflare edge (free, unchanged) ??? app.khelsutra.guru (Pages)
54                                     ??? api.khelsutra.guru ??? Cloud Run
(FastAPI control plane, scale-to-zero)
55                                     ? jobs
table (Cloud SQL Postgres)
56                                     ? signed
resumable URLs
57                                     ?
58                                     GCS buckets
(uploads / reels, lifecycle ? cold)
59                                     ? pull
jobs, outbound-only
60                                     GPU box = first
`byo_worker` (PLATFORM P3)
61     ```
62
63     - Uploads go **browser ? GCS directly** (resumable signed URLs) ? no proxy body limit,
no home
64     bandwidth for ingest; the box pulls only what it processes (WASB/owned-model jobs),
Cloud Run
65     handles Gemini-path jobs entirely in-cloud.
66     - Auth graduates to **Firebase Auth** (free; Google + email sign-in; India-friendly
phone OTP
67     later); Access allowlist retires.
68     - Post-credits estimate: Cloud Run ~$0?5 (free tier: 2 M req + 180 K vCPU-s/mo, scale-
to-zero) .
69     GCS 100 GB ? $2.60 + egress $0.12/GB (reel downloads ? watch this line) . Cloud SQL
smallest
70     ? $10 ? **~$15?20/mo**, inside budget. Egress lever if it grows: serve reels from
Cloudflare R2
71     (zero egress) while keeping everything else GCP.
72
73     ## 4. Alternatives considered
74
75     | Option | Monthly (alpha ? beta) | Pros | Cons |
76     |---|---|---|---|
77     | **Recommended hybrid** (CF edge + box ? +GCP) | **$0 ? ~$0 w/ credits** (~$15?20
after) | Cheapest at every phase; credits used where they matter; maps to PLATFORM P2/P3 | Two
vendors (CF + GCP) |
78     | **All-DigitalOcean** | ~$11 ? ~$17 ($6 droplet + Spaces $5: 250 GiB + **1 TiB egress
included**) | One simple vendor; Spaces' included egress is the best video-download deal; per-
second billing | No credits; no scale-to-zero; still needs a tunnel-equivalent to reach the box
(run `frp`/WireGuard on the droplet) |
79     | **All-GCP from day 1** | ~$18+ ? credits | Single vendor, credits immediately | LB+IAP
standing cost just to gate an alpha; no free tunnel to the home box; credits burn on plumbing |
80     | **All-Cloudflare** (Pages+Workers+R2) | ~$0 ? ~$5 | Zero egress on R2; cheapest
forever | FastAPI doesn't run on Workers (API stays on box/Cloud Run anyway); ignores GCP
credits |
81
82     ## 5. Domain setup ? GoDaddy ? Cloudflare DNS (one-time, ~15 min, you)
83
84     1. Create a free Cloudflare account ? **Add a domain** ? `khelsutra.guru` ? Free plan.
Cloudflare
85     imports existing records and shows **two nameservers** (e.g.
`xxx.ns.cloudflare.com`).

```

```

86      2. GoDaddy ? **My Products ? khelsutra.guru ? Manage DNS ? Nameservers ? Change ?
87      "I'll use my own nameservers"*** ? paste the two Cloudflare nameservers ? Save.
Propagation:
88      minutes to 24 h. (Registration/renewal stays at GoDaddy; only DNS moves. Optional
later:
89      transfer the registration to Cloudflare Registrar for at-cost .guru renewals.)
90      3. Records (I manage after takeover): `app` ? Pages project; `api` ? `

```

28. assistant (2026-06-21T02:35:02.271Z)

Now let me look at the database jobs table more carefully to understand the current structure:

29. user (2026-06-21T02:35:03.200Z)

```

188:      CREATE TABLE IF NOT EXISTS jobs (

```

30. assistant (2026-06-21T02:35:05.713Z)

Now let me search for any references to compute/GPU dispatch or external services:

31. user (2026-06-21T02:35:06.192Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/api/job_worker.py:every cross-
function seam ? `db_instance`, `global_config`, `_execute_processing`,
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/api/job_worker.py:    is the
segmenter's `_execute_processing`, run UNDER the destroy-after-confirm contract.
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/api/job_worker.py:        result =
_main._execute_processing(
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/infrastructure/database.py:
conn.execute("PRAGMA foreign_keys = ON")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/infrastructure/database.py:
conn.execute("PRAGMA busy_timeout = 30000")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/infrastructure/database.py:
conn.execute("PRAGMA journal_mode = WAL")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/infrastructure/database.py:    def
_execute_write(self, sql: str, params: Tuple[Any, ...], error_msg: str) -> bool:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/infrastructure/database.py:
conn.cursor().execute(sql, params)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/infrastructure/database.py:
cursor.execute(ddl)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/infrastructure/database.py:
version = cursor.execute("PRAGMA user_version").fetchone()[0]
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/infrastructure/database.py:
# future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA user_version = 6
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/infrastructure/database.py:
cursor.execute("""
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/infrastructure/database.py:
cursor.execute("""
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/infrastructure/database.py:
cursor.execute("""
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/infrastructure/database.py:
cursor.execute("""
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/infrastructure/database.py:
cursor.execute("PRAGMA user_version = 1")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/infrastructure/database.py:
cursor.execute("""
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/infrastructure/database.py:
cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON jobs(status)")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/infrastructure/database.py:
cursor.execute("PRAGMA user_version = 2")
```

32. assistant (2026-06-21T02:35:08.381Z)

Now let me search for Gemini provider and how AI handoff works:

33. user (2026-06-21T02:35:08.694Z)

```
1     import os
2     import time
3     import json
4     import logging
5     from typing import Tuple
6     from google import genai
7     from google.genai import types
8     from backend.providers.base import AIVideoProvider
9     from backend.infrastructure.execution_policy import (
10         ExecutionPolicy, PolicyTimeout, poll_until, run_bounded,
11     )
```

```

12
13     logger = logging.getLogger(__name__)
14
15     # The Gemini File API caps uploads at 2GB; stay well under it. Callers must chunk so
16     # each uploaded clip is below this. Oversize uploads are refused (not silently
17     # attempted).
18     MAX_UPLOAD_MB = 500
19
20     class GeminiProvider(AIVideoProvider):
21         def __init__(self, api_key: str, model_name: str):
22             super().__init__(api_key, model_name)
23             # Initialize client lazily to avoid connection/auth issues during class
24             instantiation
25             self._client = None
26
27         @property
28         def client(self):
29             if self._client is None:
30                 self._client = genai.Client(api_key=self.api_key)
31             return self._client
32
33         def _safe_delete(self, video_file) -> None:
34             """Best-effort cleanup of an uploaded File (never raises)."""
35             try:
36                 self.client.files.delete(name=video_file.name)
37             except Exception:
38                 pass
39
40         def analyze_video(self, video_path: str, prompt: str,
41                           chunk_duration: float = 0.0,
42                           label: str = "") -> Tuple[list, dict]:
43             log_prefix = f"Gemini {label}" if label else "Gemini"
44
45             metrics = {
46                 "upload_time": 0.0,
47                 "process_time": 0.0,
48                 "gen_time": 0.0
49             }
50
51             # 0. Refuse oversize uploads (Gemini File API limit is 2GB; we cap lower for
52             # safety/speed)
53             try:
54                 size_mb = os.path.getsize(video_path) / (1024 * 1024)
55             except OSError:
56                 size_mb = 0.0
57             if size_mb > MAX_UPLOAD_MB:
58                 logger.error(f"[{log_prefix}] {video_path} is {size_mb:.0f}MB >
59                 {MAX_UPLOAD_MB}MB cap ? "
60                 f"SKIPPED WITHOUT ANALYSIS (no segments for this time range). "
61                 f"Re-encode/downscale chunks (indexing.ai_chunk_scale_width) or
62                 chunk smaller.")
63                 metrics["oversize_skipped"] = True # callers aggregate this into the run
64                 report
65                 return [], metrics
66
67             # 1. Upload (retry transient network failures, e.g. WinError 10053 / reset
68             # connections)
69             logger.info(f"[{log_prefix}] Uploading {video_path}...")
70             t_upload_start = time.time()
71             video_file = None
72             for attempt in range(3):
73                 try:
74                     video_file = self.client.files.upload(file=video_path)
75                     metrics["upload_time"] = time.time() - t_upload_start
76                     logger.info(f"[{log_prefix}] Upload complete in

```

```

{metrics['upload_time']:.1f}s. URI: {video_file.uri}")
70         break
71     except Exception as e:
72         logger.warning(f"[{log_prefix}] Upload attempt {attempt + 1}/3 failed:
{e}")
73         if attempt < 2:
74             time.sleep(2 * (attempt + 1))
75     if video_file is None:
76         logger.error(f"[{log_prefix}] Upload failed after 3 attempts.")
77         return [], metrics
78
79     # 2. Wait for processing on Google servers ? a BOUNDED poll (a stuck file must
not hang
80     # the worker forever). Per-poll logs at DEBUG (no INFO spam); see
EXECUTION_POLICY.md.
81     t_process_start = time.time()
82     process_policy = ExecutionPolicy(poll_every_n_sec=10.0, max_wait_sec=1200.0) #
20 min cap
83
84     def _processed():
85         nonlocal video_file
86         video_file = self.client.files.get(name=video_file.name)
87         return None if video_file.state.name == "PROCESSING" else video_file
88
89     try:
90         video_file = poll_until(_processed, process_policy,
91                                label=f"[{log_prefix} processing", logger=logger)
92     except PolicyTimeout:
93         logger.error(f"[{log_prefix}] Processing exceeded
{process_policy.max_wait_sec:.0f}s ? giving up.")
94         self._safe_delete(video_file)
95         return [], metrics
96     except Exception as e:
97         logger.error(f"[{log_prefix}] Status check failed: {e}")
98         self._safe_delete(video_file)
99         return [], metrics
100     if video_file.state.name == "FAILED":

```

34. assistant (2026-06-21T02:35:10.936Z)

Let me get a final summary view of the database schema and look at existing tests/documentation:

35. user (2026-06-21T02:35:11.872Z)

```

def _migrate_to_v2(self, cursor):
    # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per submitted
    # /api/process request. `status` is the EXECUTION state of the job
    # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
    # that failed validation) lives inside `result` JSON as result["status"].
    # `result` is the full sync-era response dict (incl. report paths), so the
    # observability report is the job's artifact, per the plan.
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS jobs (
            id TEXT PRIMARY KEY,
            video_path TEXT NOT NULL,
            video_id TEXT,
            segmenter TEXT,
            player_pool TEXT,
            max_frames INTEGER,
            status TEXT NOT NULL DEFAULT 'queued',
            progress TEXT,
            error TEXT,
            result TEXT,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            started_at TIMESTAMP,

```

```

        finished_at TIMESTAMP
    )
    """)
    cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON jobs(status)")
    cursor.execute("PRAGMA user_version = 2")

def _migrate_to_v3(self, cursor):
    # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2). `policy` = the

```

36. assistant (2026-06-21T02:35:13.928Z)

Now let me check for any documentation about the API contract:

37. user (2026-06-21T02:35:14.310Z)

```

C:\Users\avidu\Projects\badminton-highlight-indexer\docs\ABLATION_LAYER.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\ANALYZERS_AND_RECONCILERS.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/checkpoints/2026-06-14-hardening-loop-complete.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/checkpoints/2026-06-eval-baseline\README.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/checkpoints/2026-06-logging-reporting\README.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/checkpoints/2026-06-scaffolding-complete\README.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/checkpoints/2026-06-strategy-foundation\README.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/COMMERCIAL_READINESS_REVIEW.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/decisions/DECISIONS.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/DEFERRED_HARDENING_PLAN-COMPLETED-2026-06-14.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/HARDENING_LOOP_HANDOFF-COMPLETED-2026-06-14.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/MONETIZATION_AUDIT.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/past_projects/DECOUPLED_COMPUTE/01-EXISTING-PLANS-AND-GAP.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/past_projects/DECOUPLED_COMPUTE/02-COMPUTE-MAP-AND-CONTRACT.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/past_projects/DECOUPLED_COMPUTE/03-PLATFORM-OPTIONS.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/past_projects/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/past_projects/DECOUPLED_COMPUTE/05-COST-ATTRIBUTION.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/past_projects/DECOUPLED_COMPUTE/06-RISKS-AND-OPEN-QUESTIONS.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/past_projects/DECOUPLED_COMPUTE/07-COMPUTE-ABSTRACTION.md

```

38. assistant (2026-06-21T02:35:16.061Z)

Perfect! Let me read the compute map and cost attribution documents:

39. user (2026-06-21T02:35:16.333Z)

```

1      # 02 · Compute map & data contract
2
3      **Status:** research synthesis, code-verified 2026-06-20 · **Date:** 2026-06-20
4
5      > What actually consumes GPU/CPU/WSL/external-API across the three repos, ranked by how
hard each is to
6      > lift into a Linux cloud-GPU container ? plus the one data contract everything keys on.

```

```

7
8     ## 1. Compute footprint of `badminton-highlight-indexer` (the consumer/trainer)
9
10    | Component | Needs | Portability risk ? cloud Linux GPU |
11    |---|---|---|
12    | **`gemini` segmenter** (default) | external paid API (`GEMINI_API_KEY`) | **LOW.**
Pure HTTP; no GPU. Silently returns **zero** segments without the key ? inject as a secret. |
13    | **`motion` segmenter** | CPU (OpenCV frame-diff) | **NONE.** Pure CPU, no API key. |
14    | **`yolo_hybrid` segmenter** | GPU (torchvision Faster R-CNN + ByteTrack); auto CPU-
fallback | **LOW.** Pure-pip; runs on CPU or CUDA. Pin `supervision<0.30` (ByteTrack API
drifts). Pre-bake COCO weights into the image to avoid first-run hub pulls. |
15    | **`trajectory_hybrid` / TrackNet / WASB** | **GPU + WSL2 bridge** (TensorFlow);
hardcoded `~/models` paths, hardcoded `device='cuda'` (no CPU fallback); **un-vendored**
research repos; TrackNet healthcheck requires a visible GPU | **HIGH ? but the swap seam is
already built.** Runs *only* via `wsl -d Ubuntu ? conda activate` **on Windows** ? on a Linux
GPU box WSL is **dropped, not ported** (the conda env runs natively). The `DetectorRunner` ABC
([`detectors/base.py`](../backend/pipeline/detectors/base.py)) exists *expressly* to de-risk
this: WSL plumbing is a separate `WslCommandMixin` (a native runner must not inherit it),
Windows behavior is **100% preserved,** and the **parity contract is the `Frame,Visibility,X,Y`
CSV** ? any runner emitting it reuses `parse_trajectory_csv` + `trajectory_to_action_windows`
verbatim, so everything downstream is byte-identical. The `LinuxNativeRunner` impl doesn't exist
yet / has never been run ? prove it via the **M3.5 parity test on DigitalOcean** ([04](04-BUILD-
AND-TEST-PLAN.md)). |
16    | **`fusion_hybrid` / Gen-0 head** | CPU/tiny GPU (numpy/scipy) | **NONE.**
[`training/gen0/harness.py`](../training/gen0/harness.py): ~1M params, ~4.5 GB VRAM, trains
in *minutes*. This is the cheap retrain target. |
17    | **`ffmpeg` chunking / stitching** | CPU (`libx264`) | **LOW.** No nvenc/driver-codec
dependency (a portability win) ? `apt-get install ffmpeg` is enough. |
18    | **Gen-2 VLM** (ms-swift / QLoRA Qwen3-VL) | **multi-GPU** (6?8-GPU GRPO reference; 24
GB VRAM floor) | **OUT OF SCOPE for $200.** **"Largest open risk, currently unquantified." Do
not attempt on this budget. |
19
20    **Two structural container blockers beyond WSL:**
21    - **TF-vs-torch runtime conflict.** TrackNet is TensorFlow; the rest is PyTorch. Their
pip-bundled CUDA
22    userspaces disagree ? "works alone, segfaults together." Fix = **two single-
framework images** (a torch
23    image, a TF image) sharing a `/scratch` volume. So the container topology itself is
non-trivial.
24    - **CUDA wheel selection.** `torch>=2.12.0` pinned with no index ? the
Dockerfile/installer must pick
25    `--index-url ?/cu124` vs `?/cpu` explicitly.
26
27    **Path / I/O blockers (confirmed live):**
28    - Hardcoded literal `"output"` in
[`trajectory_hybrid.py:196,272`](../backend/pipeline/segmenters/trajectory_hybrid.py) ? not
redirected by `--output-dir`.
29    - Local-file assumptions: `cv2.VideoCapture`, `ffmpeg -i <path>`, whole-file MD5,
`allowed_video_root`
30    check in [`main.py`](../backend/main.py). No URL/stream path ? needs a
`fetch(uri)?local` + `put` shim.
31
32    ## 2. Compute footprint of `sports-data-collector` (the producer/curator)
33
34    CPU-first; "the gap is narrow." Only GPU touchpoint is **opt-in `nvenc`** (`--encoder
nvenc`, commit #28)
35    for proxy transcode + batch transcode ? defaults to CPU `libx264`. Shares the same
local-filesystem + MD5
36    assumptions. The export **`manifest.json` omits an `md5` field** (`proxy.py` carries
only `source_md5` /
37    `proxy_md5`); the collector keys by **human `video_id`** (`curate.py`), while the
indexer keys by **file
38    MD5** ? see §4.
39
40    ## 3. Compute footprint of `rally-annotator` (the labeler)

```

```

41
42     Effectively **no heavy compute** ? a VLC 3.x Lua extension a human drives to mark rally
`start`/`end` +
43     ending reason ? `<video>.rallies.csv`. Nothing to containerize; it's the *source* of the
labels that must
44     land in the bucket.
45
46     ## 4. The one data contract everything keys on
47
48     ```
49     rally-annotator                sports-data-collector                badminton-highlight-indexer
50     (label by hand) ?????????? (curate + license + export) ???
(validate?segment?index?stitch?eval?train)
51     <video>.rallies.csv          SQLite ? manifest.json + CSVs          keyed by file MD5
52     ```
53
54     - **Identity:** the indexer keys videos by **whole-file MD5** (`compute_video_id`, 64
KB-chunked); the
55     collector keys by **human `video_id`**. The manifest's `local_path` ties them
together. **Any re-encode
56     changes the MD5** and silently breaks the join.
57     - **Intervals:** temporal `[start, end)` in **decimal seconds**; ending-reason taxonomy
58     (winner/forced_error/unforced_error/service_fault/let/other) is the cross-sport label
set.
59     - **Train/serve wall (ADR-008):** the boundary between training and serving is an
**artifact bundle**
60     (weights + manifest), **never a code import** ? keyed by content/semver.
61
62     ### What this means for a bucket workflow
63
64     1. **Add `md5` to the collector export manifest** (or decide *original-vs-proxy* keying)
**before** feeding
65     a bucket ? otherwise the MD5-keyed consumer can't resolve labels?video. *This is an
M2 prerequisite, not
66     a footnote.*
67     2. **Store byte-identical files** the labels were made against. If you must proxy-
transcode for size, carry
68     an explicit `original_md5` ? `proxy_md5` mapping so the join survives.
69     3. A cloud `train`/`infer` job's **inputs** = `{video object(s), label CSV(s),
manifest.json}` from the
70     bucket; its **outputs** = `{versioned weight bundle}` (train) or `{highlights.mp4,
intervals.csv,
71     report.json}` keyed by `video_id` (infer). These map cleanly onto the existing per-
video output layout.
72
73     ### Inputs/outputs a cloud job must exchange (summary)
74
75     | Verb | Pulls from bucket | Pushes to bucket |
76     |---|---|---|
77     | `train` | `videos/`, `labels/`, `manifest.json` (+ latest checkpoint to resume) |
`weights/<semver>/{model, manifest}` + periodic `weights/checkpoints/` |
78     | `infer` | one `videos/<md5>.mp4` | `outputs/<video_id>/{highlights.mp4, intervals.csv,
report.json}` |
79

```

40. user (2026-06-21T02:35:16.621Z)

```

1     # 05 · Cost attribution ? per-video inference & per-run training billing
2
3     **Status:** PROPOSAL (net-new design) · **Date:** 2026-06-20
4
5     > **Side goal:** a non-owner user runs a job and gets the **exact billing for that
video** (inference), and
6     > training cost can be **segmented similarly** (per run, optionally amortized per
contributed video). This

```



```

7      > is wholly new ? the `jobs` table has **no cost columns** today
([`database.py:188`](../backend/infrastructure/database.py)).
8      > The good news: a job is already the unit of work, and serverless platforms already
meter per invocation ?
9      > so per-video billing is mostly *bookkeeping on top of seams that exist.*
10
11     ## 1. The core idea
12
13     **A job is the billing unit.** An `infer` job is **1:1 with a video** (keyed by
`video_id`/MD5), so the
14     job's cost *is* the per-video bill. A `train` job is **1:1 with a training run**, so its
cost is the run
15     cost ? which we then optionally **split across the videos that fed it**.
16
17     ```
18     cost_usd(job) = ?_resource usage(job, resource) × rate(resource, pricebook_version)
19     ```
20
21     Resources to meter: **GPU-seconds** (the big one), CPU-seconds, wall-seconds, **egress
GB** (reel
22     downloads ? the line `HOSTING_PLAN.md` flags), storage GB-hours, and **external-API
calls** (Gemini
23     tokens/calls ? the provider already returns metrics: `analyze_video` ? (segments,
metrics)).
24
25     ## 2. Metering mechanics (cheapest accurate source first)
26
27     | Source of GPU-seconds | When | Accuracy |
28     |---|---|---|
29     | **Serverless per-invocation report** (RunPod Serverless / Cloud Run / Modal) | M3
inference endpoint | **Best** ? the platform bills exact GPU-seconds per job; that number *is*
the bill. **This is the strongest argument for a serverless `infer` topology**
([03](03-PLATFORM-OPTIONS.md)). |
30     | **Worker-measured `gpu_active_seconds`** (timestamp around the GPU section, × the
known `gpu_type` rate) | DO Droplet / VM where there's no serverless meter | Good ? separate
`gpu_active_seconds` from `wall_seconds` so you don't bill the user for the bucket-sync/upload
time. |
31     | **Provider billing API reconciliation** (monthly) | all | Truth source ? reconcile the
sum of job costs against the invoice; carry a `reconciliation_factor`. |
32
33     Egress is counted from bytes downloaded (R2 = zero egress ? simplest). Gemini cost comes
from the metrics
34     the provider already returns.
35
36     ## 3. Schema delta (extends the existing `jobs` table; no new datastore)
37
38     Add via the existing `_add_column_idempotent` / `PRAGMA user_version` migration ladder:
39
40     ```sql
41     ALTER TABLE jobs ADD COLUMN owner_id          TEXT;      -- who pays (the non-owner user /
tenant)
42     ALTER TABLE jobs ADD COLUMN compute_backend TEXT;      -- runpod_serverless |
gcp_cloudrun | do_droplet ?
43     ALTER TABLE jobs ADD COLUMN gpu_type          TEXT;      -- L4 | A4000 | T4 ?
44     ALTER TABLE jobs ADD COLUMN gpu_active_seconds REAL;
45     ALTER TABLE jobs ADD COLUMN wall_seconds      REAL;
46     ALTER TABLE jobs ADD COLUMN bytes_out         INTEGER;   -- egress
47     ALTER TABLE jobs ADD COLUMN gemini_cost_usd   REAL;
48     ALTER TABLE jobs ADD COLUMN pricebook_version TEXT;
49     ALTER TABLE jobs ADD COLUMN cost_usd          REAL;      -- computed total
50     ALTER TABLE jobs ADD COLUMN cost_breakdown_json TEXT;    -- {gpu: .., egress: .., api:
..}
51
52     -- price book (versioned; rates change, and differ per provider/gpu_type)
53     CREATE TABLE IF NOT EXISTS pricebook (

```

```

54     version TEXT, resource TEXT, gpu_type TEXT, unit_price_usd REAL, currency TEXT,
effective_at TIMESTAMP
55 );
56
57 -- training-run cost attribution across contributing videos
58 CREATE TABLE IF NOT EXISTS training_contributions (
59     training_job_id TEXT, video_id TEXT, weight REAL    -- weight = clip-seconds share (or
1/N)
60 );
61 ```
62
63 The pricebook is seeded from the same provider-grounded GPU set as the device
profiles**
64 ([07](07-COMPUTE-ABSTRACTION.md) §5) ? D0/Paperspace now (A4000 / L40S / A100), GCP
later (T4 / L4 / A100) ?
65 so one {provider, gpu_type} table carries both the $/hr rate (here) and the device-
tuning knobs (there)**,
66 keyed off the gpu_type that run_telemetry's nvidia-smi probe already yields.
67
68 owner_id is the per-tenant scoping the multi-tenant model
([PLATFORM_ARCHITECTURE.md](../PLATFORM_ARCHITECTURE.md) §4)
69 will need anyway ? adding it now keeps "a non-owner user is billed for their jobs" a
first-class property.
70
71 ## 4. Per-video inference billing
72
73 1. An infer job's cost_usd + cost_breakdown_json is the per-video bill (job is
1:1 with the video).
74 2. Expose GET /api/jobs/{id}/cost and a per-video cost history (GET
/api/videos/{video_id}/cost), scoped
75 by owner_id.
76 3. Optional resale markup: a margin config ? price_to_user = cost_usd × (1 +
margin); store both so the
77 user sees a transparent breakdown and you keep your margin auditable.
78
79 Example breakdown (illustrative): 3-min L4 inference + a 40 MB reel download + one
Gemini call ?
80 gpu: 180 s × $0.00022/s = $0.040 + egress: 0.04 GB × $0 (R2) = $0 + gemini: ~$0.05`
? ~$0.09/video.
81
82 ## 5. Per-training cost segmentation
83
84 1. A train job's cost_usd is the run cost (e.g. Gen-0 ? a few cents; a future
VLM run would be $-tens).
85 2. To "segment similarly," attribute the run cost across the N videos/labels in that run
via
86 training_contributions ? equal split (1/N) or weighted by clip-seconds
contributed. This
87 yields "cost per video contributed to training," the symmetric analog of the per-
video inference bill.
88 3. Record training_job_id on the produced weight bundle so a served model's lineage
carries its training
89 cost (useful for amortizing training into a per-inference price later, if you
choose).
90
91 ## 6. Trust & reconciliation
92
93 - Transparency: always show the user the cost_breakdown_json +
pricebook_version, not just a total.
94 - Reconciliation: metered cost ? provider invoice but not exact (overhead, rounding,
idle). Reconcile
95 monthly: reconciliation_factor = invoice_total / ? metered_cost; surface drift if it
exceeds a few %.
96 - Spend guard: a per-owner_id rate limit + a global hard cap (the same cap that
protects the $200)

```

```
97         means a leaked API key can't run up someone else's bill or drain the budget.
98
99     ## 7. Why this is cheap to build
100
```

41. assistant (2026-06-21T02:35:19.071Z)

Excellent! Now let me check the CORS and auth status to confirm the gap:

42. user (2026-06-21T02:35:20.080Z)

```
20:from fastapi.middleware.cors import CORSMiddleware
38:# Enable CORS for the local web interface. allow_origins='' with allow_credentials=True is
40:# auth/tenancy work; this tool uses no cookies, so credentials stay off.
42:    CORSMiddleware,
44:    allow_credentials=False,
46:    allow_headers=["*"],
128:    """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
263:    allowed_root = getattr(getattr(global_config, "security", None), "allowed_video_root",
None)
```

43. assistant (2026-06-21T02:35:22.298Z)

Perfect. Now I have a comprehensive view. Let me compile the structured output:

44. user (2026-06-21T02:36:13.470Z)

Structured output provided successfully

45. assistant (2026-06-21T02:36:24.974Z)

Summary

I've completed a comprehensive mapping of the **control plane + hosted-service surface** for the badminton-highlight-indexer. Here are the key findings:

What Already Exists (Core Contract)

- **FastAPI endpoints**: `/api/process` (submit job), /api/jobs/{job_id}` (poll), /api/query`/api/compile`/api/highlights` (segment filtering, stitching, download)`
- **Chunked upload**: browser ? server assembly (?95 MB parts, 4 GB max)
- **Jobs table** (v2?v4 schema): id, video_path, segmenter, status, progress, error, result (full response JSON), user_id (nullable), policy (JSON ExecutionPolicy), next_poll_at
- **Async job worker**: `ThreadPoolExecutor` (max_workers=1) drains jobs via atomic`claim_due_job()` CAS, runs pipeline, records outcome`
- **Execution policy framework**: `ExecutionPolicy` class, run_bounded()` (per-attempt timeout + retries), poll_until(), JobWaiting` exception for cooperative rescheduling`

AUTH Gap (CORS *****, No Identity)

- CORS configured `allow_origins=['*']`, allow_credentials=False``
- No endpoint requires authentication; all endpoints open
- Path validation (`security.allowed_video_root``) is advisory only
- **Required for hosting**: extract Cloudflare Access header (`Cf-Access-Authenticated-User-Email``) or Firebase Auth token at every protected endpoint

Per-Video Cost Ledger Gap (NET-NEW)

The `jobs`` table has **zero cost columns**. Per-video billing requires:

- `owner_id`` (tenant scoping for multi-user)
- `compute_backend`` (runpod_serverless|gcp_cloudrun|do_droplet)
- `gpu_type`` (L4|T4|A4000)
- `gpu_active_seconds`, wall_seconds`, bytes_out`` (egress)
- `gemini_cost_usd`, pricebook_version`, cost_usd`, cost_breakdown_json`` ({gpu: .., egress: .., api: ..})
- New tables: `pricebook`` (versioned rates) and `training_contributions`` (cost split across

training videos)

Cloud Run + GPU Dispatch Gap (NET-NEW)

Currently segmentation runs **in-process** on the FastAPI worker thread. For the beta topology:

- Control plane (Cloud Run) receives `/api/process`` submit
- Must dispatch to GPU worker (home box or cloud container) via HTTP poll or Cloud Task
- Worker measures `gpu_active_seconds`` (timestamp probes during GPU code) and `wall_seconds``
- Returns results + metrics; control plane aggregates into `cost_usd``
- **No dispatch code exists** ? the `JobWaiting`` + `ExecutionPolicy`` seam supports it perfectly; just needs cloud-specific worker URL routing

Segment vs. Stitch Split (Unclear)

Stitching (ffmpeg, CPU-only, ~3 min per reel) currently runs **synchronously** in `/api/compile``. Design decision needed: stay in Cloud Run (sync, cheapest) or move to job queue (if users batch).

Reuse Potential

`ExecutionPolicy``, `run_bounded()``, `poll_until()``, `JobWaiting``, and the jobs table + `claim_due_job()`` CAS all carry **directly** into a Cloud Run + GPU dispatch model. Gemini provider already returns structured metrics. The schema is ready for a v6 migration adding cost columns.